

DM42 Applied Problem Solving

**Scientific, Mathematical & Practical Techniques for the
SwissMicros DM42**

Volume I · Applications

September 2026 · First edition

A problem, a model, a sequence of operations, and a result worth checking.

Preface

The DM42 is particularly rewarding when a calculation has structure: a physical model, several related quantities, a fit to observations, or a numerical answer that deserves a check. This book starts at that point. It assumes ordinary RPN fluency and elementary algebra, calculus, complex numbers and vectors. Each chapter introduces a useful situation, supplies the relevant mathematics, and shows how to carry the work through on the calculator. The separate Volume II is reserved for programming.

Values are given with units in the prose and equations. A DM42 number, by itself, does not carry physical units. Keep a written unit beside each stored quantity; entering 47 when the model requires $47 \mu F$ changes a time constant by a factor of a million. The displayed number of digits is a choice of format, not an assertion about measurement precision.

Basis for calculator instructions. SwissMicros *DM42 User Manual*, v3.42, dated 22 September 2026; SwissMicros *DM42 Quick Reference Guide*, v3.5.5 (Free42 3.2.8 extensions), dated 22 April 2025; SwissMicros *DM42 Functions*, v2.0.2, dated 22 April 2025. The SwissMicros DM42 firmware listing available at preparation time identifies DM42 program 3.26 and DMCP 3.29 (14 November 2025). The author's Free42 documentation, updated 23 April 2025 for extensions, provides secondary detail. These examples describe the **DM42**, with ordinary four-level RPN stack mode. Software revisions can change menus; use the built-in Help when a label differs. Documentation was checked; no physical DM42 was available to actuate every key.

Reading a procedure

- [ENTER], [STO], [RCL], [×]: physical key. A number in upright type is an input to type, with [ENTER] explicitly separating stack operands when required.
- {STAT}, {CFIT}, {LINF}: an application or soft-menu label; press the key immediately below it. → between braces denotes movement through menus, not an arithmetic operation. {▼} moves to the next menu page.
- f SDEV: execute the named function **SDEV** through CATALOG → FCN, or use its corresponding shifted key when that key is known. The same rule applies to f PI, f SIN, f X², and other named operations. This notation makes every shifted operation explicit without assuming that its name is a primary key. A sequence such as 12 [ENTER] 3 [÷] reads as one continuous calculation.
- X: / Y:: stack register contents. →POL, CLΣ, Σ+, f f(x) are **actual calculator function/menu spellings** in the SwissMicros guide. Mathematical $\sqrt{x}$ and $\sum$ are typeset mathematical notation, not necessarily literal screen strings.
- DM42 rectangular complex display puts an i before the imaginary component, for example 0.2343 i1.4456; its polar display uses the calculator's ∠, for example 1.4645 ∠80.7937. Conventional mathematical $r \angle \theta$ in an equation is distinguished from the LCD spelling. ▲ and ▼ identify navigation/annunciator glyphs.

Unless stated otherwise, use decimal-point radix, degrees for geometric angles, radians when the variable of integration is an angle, and clear statistical sums before entering a new dataset. For every sequence, the formulas state stack operand order. Intermediate calculations may be made by recalling a stored value instead of retyping it. All amounts in rand are examples, not forecasts.

Contents

Chapter	Page
1. Calculations with memory and scale	4
2. Geometry and trajectories	5
3. Motion, rotation and orbits	6
4. Oscillations, gases, optics and light	7
5. Direct-current electronics	9
6. Complex numbers and alternating current	10
7. Matrices and coupled systems	11
8. Numerical root finding	12
9. Numerical integration	14
10. Experimental data and regression	15
11. Probability and measurement uncertainty	16
12. Finance and workshop mathematics	17
13. Mixed applications and numerical judgment	18
Addendum. DM42 for Programmers	21
Appendix A. Techniques at a glance	37
Appendix B. Constants and documentation	38
Alphabetical index	39

1. Calculations with memory and scale

The stack is ideal for a transient expression. A named variable or numbered register is better for a measured parameter that will be reused. The DM42 initially provides numbered registers 00–24; the statistics functions use a block beginning at R11 by default, so this book reserves R00–R09 for hand calculations. STO 00 records X, and RCL 00 brings a copy back to the stack. A named variable such as TAU can be stored by [STO] [ENTER] TAU [ENTER]; the manual limits names to seven characters. Recalling such a variable is useful when several chapters use the same apparatus settings.

1.1 Calibrate an audio level change

Problem. An audio sensor produces 0.60 mV_{rms} before amplification and 2.40 mV_{rms} after it. Report voltage gain in decibels, assuming equal impedances at the two measurement points.

Model. The amplitude ratio is $A_v = V_2 / V_1 = 4$, and the amplitude-level difference is

$$\Delta L = 20 \log_{10} \left(\frac{V_2}{V_1} \right) = 12.04119983\text{ dB}.$$

The factor 20 follows because power is proportional to squared voltage **at equal resistance**; it is not the formula for a power ratio measured independently. A 6.02 dB voltage increase is approximately a factor of two.

On the DM42. 0.60 [ENTER] 2.40 f X<>Y [÷] would divide in the wrong order if the swap were omitted: with $Y = 0.60$ and $X = 2.40$, [÷] returns 0.25. Instead enter 2.40 [ENTER] 0.60 [÷] for 4; then f LOG 20 [×]. If LOG is reached through a shifted key on your layout, invoke the named function LOG. Result: **12.04 dB**. As a check, $20 \log_{10} 2 \approx 6.02$, and doubling twice gives 12.04.

Instrument note. In a chain of gains and attenuations, add dB values, but multiply their underlying linear gains. Do not add a dB value to a voltage.

1.2 Use LASTX to keep a difficult denominator

Problem. A sensor voltage divider has $R_T = 10.0\text{ k}\Omega$, $R_B = 4.70\text{ k}\Omega$, and a $10.0\text{ k}\Omega$ meter connected across R_B . Determine the loaded divider voltage for $V_S = 3.30\text{ V}$.

Model. $R_E = R_B \parallel R_M = R_B R_M / (R_B + R_M) = 3.197278912\text{ k}\Omega$; hence

$$V_o = V_S \frac{R_E}{R_T + R_E} = 0.7994845361\text{ V}.$$

On the DM42. Work in $\text{k}\Omega$ throughout: 4.7 [ENTER] 10 [×] 4.7 [ENTER] 10 [+] [÷] leaves $R_E = 3.197278912$ in X. [STO] 00 saves it. Now 10 [RCL] 00 [+] [RCL] 00 f X<>Y [÷] 3.3 [×] gives 0.79948 V : after the swap, $Y = R_E$ and $X = 10 + R_E$. An easier alternative is 3.3 [ENTER] [RCL] 00 [×] 10 [RCL] 00 [+] [÷]. The latter avoids relying on the prior stack state.

Without the meter, $V_o = 3.30 (4.7 / 14.7) = 1.0551\text{ V}$. The load reduces the indication by about 24%. When repeating with several meter resistances, store the fixed top and bottom resistors and recompute the parallel

combination. **LASTX** can restore the last X operand after many numeric operations, but it is not a general undo history; a saved register is safer for the original measurements.

1.3 Scale before entering scientific quantities

A 1.5 nF capacitor and a $2.2\text{ M}\Omega$ resistor have $\tau = RC = 3.30\text{ ms}$, because $10^{-9}10^6 = 10^{-3}$. Enter $1.5\text{ EEX } 9\text{ [+/-] [ENTER] } 2.2\text{ EEX } 6\text{ [x]}$ using the physical exponent-entry and sign-change keys; X is 0.0033 seconds. The *sign change* applies to the exponent during entry. EEX is a key operation, not multiplication by a power of ten typed into the display. $\text{DISP} \rightarrow \text{ENG}$ is convenient when comparing nF , μF and ms , but it does not convert units automatically.

Sanity check. Nanofarads times megohms produce milliseconds. Whenever a result differs by an exact factor of 10^3 or 10^6 , inspect prefix conversions before blaming the numerical method.

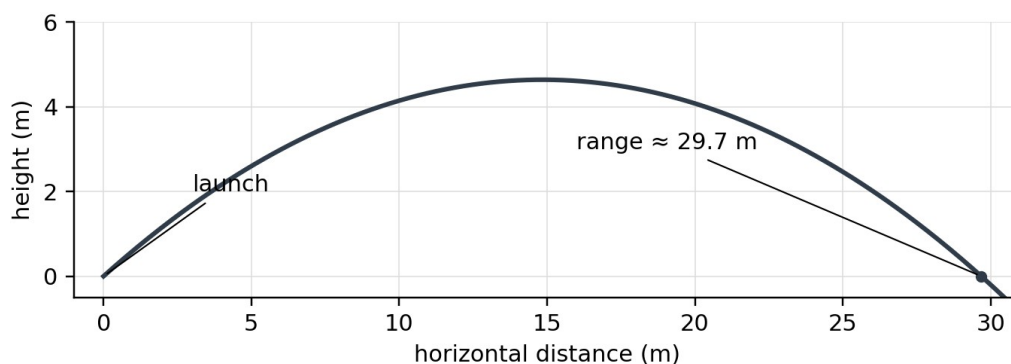
2. Geometry and trajectories

2.1 A projectile launched from level ground

Problem. A ball leaves a launcher at $v_0 = 18.0\text{ m s}^{-1}$ and $\theta = 32.0^\circ$ above horizontal. Neglect drag and assume launch and landing have the same height. Find time aloft, horizontal range, and peak height. The result is a first estimate, not a ballistics prediction in wind.

Model. Resolve $v_x = v_0 \cos \theta = 15.2649\text{ m s}^{-1}$ and $v_y = v_0 \sin \theta = 9.53855\text{ m s}^{-1}$. At touchdown, $0 = v_y t - g t^2 / 2$, with nonzero solution $t = 2 v_y / g$. Then

$$t = 1.94466\text{ s}, R = v_x t = \frac{v_0^2 \sin(2\theta)}{g} = 29.68494\text{ m}, h = \frac{v_y^2}{2g} = 4.63730\text{ m}.$$



Idealised trajectory for the stated launch speed and angle.

On the DM42. Set $\text{MODES} \rightarrow \text{DEG}$. Compute $18\text{ [ENTER] } 32\text{ f SIN [x] [STO] } 00$ to save v_y in R00. Time: $2\text{ [RCL] } 00\text{ [x] } 9.81\text{ [÷]}$, store as R01. Compute $18\text{ [ENTER] } 32\text{ f COS [x] [RCL] } 01\text{ [x]}$ for range. For height: $\text{[RCL] } 00\text{ f X}^2\text{ 2 [ENTER] } 9.81\text{ [x] [÷]}$. Named functions SIN, COS, X^2 may be called from their keys or from $\text{CATALOG} \rightarrow \text{FCN}$.

Results. $t \approx 1.94 \text{ s}$, $R \approx 29.7 \text{ m}$, $h \approx 4.64 \text{ m}$. Check the range independently with $v_0^2 \sin 64^\circ / g$. The range formula requires equal launch and landing heights. At fixed speed it is symmetrical under $\theta \mapsto 90^\circ - \theta$; this symmetry disappears if the target is elevated.

Try it. Recalculate at 58° . The range is the same idealised value, while flight time and peak height change.

2.2 Rotate a measured vector

Problem. A workshop drawing gives coordinates $(12, 5) \text{ mm}$ in a fixture frame. The fixture is rotated counterclockwise by 37° relative to machine axes. Find the machine-frame point.

Model. A *passive change of coordinates* and an *active rotation of a point* have opposite signs. Here we actively rotate the point:

$$\begin{pmatrix} x' \\ y' \end{pmatrix} = \begin{pmatrix} \cos \theta & -\sin \theta \\ \sin \theta & \cos \theta \end{pmatrix} \begin{pmatrix} 12 \\ 5 \end{pmatrix} = \begin{pmatrix} 6.57455 \\ 11.21496 \end{pmatrix} \text{ mm}.$$

On the DM42. Set degrees. Evaluate $12 [\text{ENTER}] 37 \text{ f COS } [\times] 5 [\text{ENTER}] 37 \text{ f SIN } [\times] [-]$ for x' .

Evaluate $12 [\text{ENTER}] 37 \text{ f SIN } [\times] 5 [\text{ENTER}] 37 \text{ f COS } [\times] [+]$ for y' . Store one component before computing the other. For many points sharing the same angle, compute and store $\sin \theta$ and $\cos \theta$ once. Verify $x'^2 + y'^2 = 12^2 + 5^2 = 169 \text{ mm}^2$: a rotation preserves length.

DM42 insight. The CONVERT menu's $\rightarrow \text{POL}$ and $\rightarrow \text{REC}$ functions can convert rectangular and polar coordinate pairs, but the algebra above fixes the chosen rotation sign unambiguously. Do not interpret a complex number in POLAR display as though it has physically moved; changing RECT/POLAR changes representation, not the underlying vector.

2.3 Locate a centre of mass

A 0.40 kg bearing is at $x = 0$, a 0.70 kg drive unit at $x = 0.22 \text{ m}$, and a 0.30 kg end stop at $x = 0.48 \text{ m}$ along a light rail. The centre of mass is

$$x_{cm} = \frac{\sum m_i x_i}{\sum m_i} = \frac{0.40(0) + 0.70(0.22) + 0.30(0.48)}{1.40} = 0.212857 \text{ m}.$$

On the DM42, accumulate the moment $\sum m_i x_i$ as $0.7 [\text{ENTER}] 0.22 [\times] 0.3 [\text{ENTER}] 0.48 [\times] [+]$, then divide by 1.4 . The answer lies between the extreme positions and is close to the heavier middle component. If a rail has appreciable mass, its own mass at its midpoint must be included; the point-mass model otherwise underestimates the contribution from the rail. This is a convenient place for **STO+ 00**: initialise **R00** to zero once and add each component's moment to it, but check the accumulated units are kg m before dividing by total mass.

3. Motion, rotation and orbits

3.1 Capture two carts: conserve momentum, inspect energy

Problem. A 0.30 kg cart moving at $+4.0 \text{ m s}^{-1}$ sticks to a 0.50 kg cart moving at -1.0 m s^{-1} . Find final velocity and kinetic energy converted to other forms. The impact time is short enough to neglect external impulse.

$$v_f = \frac{m_1 v_1 + m_2 v_2}{m_1 + m_2} = 0.875 \text{ m s}^{-1}.$$

On the DM42, do the signed momentum calculation explicitly: 0.3 [ENTER] 4 [×] 0.5 [ENTER] 1 [+/-] [×] [+] 0.8 [÷]. Store v_f in R00. Initial energy is 2.65 J; final energy is 0.30625 J; the difference is 2.34375 J. For the latter calculate 0.8 [ENTER] [RCL] 00 f X² [×] 2 [÷]. Momentum is conserved by this model; kinetic energy is not. If the computed final energy exceeds the initial energy for a passive sticking impact, revisit the signs and inputs.

3.2 A uniform disk flywheel

Problem. A 0.50 kg uniform disk of radius 0.15 m turns at 3000 rpm. Estimate stored rotational energy. Assume a rigid disk and ignore hub and shaft inertia.

$$I = 1/2 m r^2 = 0.005625 \text{ kg m}^2, \omega = 2\pi \frac{3000}{60} = 314.159 \text{ rad s}^{-1}, E = 1/2 I \omega^2 = 555.165 \text{ J}.$$

On the DM42. 0.5 [ENTER] 0.15 f X² [×] 2 [÷] [STO] 00 saves inertia. Evaluate 3000 [ENTER] 60 [÷] 2 [×] f PI [×] [STO] 01 to save angular speed. Then 0.5 [RCL] 00 [×] [RCL] 01 f X² [×]. If n is doubled, energy quadruples. If the disk is instead a thin ring of equal mass and radius, use $I = m r^2$ and the energy doubles. The physics, not the calculator, chooses the correct geometry.

3.3 A circular low-Earth orbit

Problem. Estimate orbital speed and period 400 km above a spherical Earth. Use $\mu = G M = 3.986004418 \times 10^{14} \text{ m}^3 \text{ s}^{-2}$ and illustrative mean radius $R_E = 6371 \text{ km}$. Neglect drag, oblateness and Earth rotation.

$$r = 6.771 \times 10^6 \text{ m}, v = \sqrt{\frac{\mu}{r}} = 7.67260 \text{ km s}^{-1}, T = 2\pi \sqrt{\frac{r^3}{\mu}} = 92.4143 \text{ min}.$$

On the DM42. Work in SI and store radius: 6371 [ENTER] 400 [+] 1000 [×] [STO] 00. Type 3.986004418 EEX 14 [RCL] 00 [÷] f SQRT for speed. For period, calculate the square-root factor first, then multiply by $2\pi/60$. The exact order is RCL 00 [ENTER] 3 [Y^X] 3.986004418 EEX 14 [÷] f SQRT 2 [×] f PI [×] 60 [÷].

Check. Distance around the circular orbit is $2\pi r \approx 42,543 \text{ km}$; divide by about 7.67 km s^{-1} to obtain about 92 minutes. Escape speed at the **same radius** is $\sqrt{2} v \approx 10.851 \text{ km s}^{-1}$, without accounting for atmospheric losses.

4. Oscillations, gases, optics and light

4.1 Estimate a mechanical resonance

A 0.25 kg mass on an ideal spring of stiffness 40 N m^{-1} has $f = (2\pi)^{-1} \sqrt{k/m} = 2.01317 \text{ Hz}$. Type 40 [ENTER] 0.25 [÷] f SQRT 2 [÷] [ENTER] f PI [÷] (left-to-right division) to obtain the frequency. Independently, $T = 2\pi \sqrt{m/k} \approx 0.4967 \text{ s}$, and $f T = 1$. The formula concerns free, small-amplitude oscillation; damping and drive can shift the observed resonance. If the mass is quadrupled, frequency halves.

4.2 Gas pressure in a rigid container

Problem. A rigid 30.0 L vessel contains 1.20 mol of an ideal gas at 298 K. Find absolute pressure. The vessel has uniform temperature, and the gas is assumed ideal.

$$p = \frac{nRT}{V} = \frac{1.20(8.314462618)(298)}{0.0300} = 99.1084 \text{ kPa}.$$

Enter 1.2 [ENTER] 8.314462618 [×] 298 [×] 0.03 [÷] 1000 [÷]. Result **99.1 kPa absolute**, close to ordinary atmospheric pressure. Atmospheric *gauge* pressure would be near zero if ambient pressure were 99.1 kPa; do not confuse the two. At fixed n, V , doubling the absolute temperature doubles pressure. A change from 25°C to 50°C is not a doubling of Kelvin temperature.

4.3 Focus an object with a thin lens

Problem. An object is 0.30 m before a thin converging lens of focal length +0.12 m. Under the paraxial approximation, find image distance and transverse magnification.

$$\frac{1}{f} = \frac{1}{s_o} + \frac{1}{s_i}, s_i = \left(\frac{1}{0.12} - \frac{1}{0.30} \right)^{-1} = 0.20 \text{ m}, m = -\frac{s_i}{s_o} = -\frac{2}{3}.$$

On the DM42. 0.12 f 1/X 0.30 f 1/X [-] f 1/X gives $s_i = 0.20$ m. Then 0.20 [ENTER] 0.30 [÷] [+/-] gives $m = -0.667$. If the sign convention changes, reconsider the equation before changing signs in the calculator. A negative magnification indicates an inverted real image under this convention. Substitute $1/0.12 = 1/0.30 + 1/0.20$.

4.4 Blue-light photon energy

At wavelength 450 nm in vacuum, $E = hc/\lambda = 4.414 \times 10^{-19} \text{ J} = 2.75520 \text{ eV}$. Use the *exact SI definitions* $h = 6.62607015 \times 10^{-34} \text{ J}\cdot\text{s}$, $c = 299792458 \text{ m}\cdot\text{s}^{-1}$, and $1 \text{ eV} = 1.602176634 \times 10^{-19} \text{ J}$. Enter hc in SI, divide by 450 EEX 9 [+/-], then divide by 1.602176634 EEX 19 [+/-]. The inverse proportionality means that halving wavelength doubles photon energy. This is single-photon energy, not a beam's total energy.

4.5 An isotope as a logarithmic clock

For a sample whose activity is proportional to remaining parent nuclei, a half-life of 5730 years gives $N(t)/N_0 = 2^{-t/5730}$. After 12,000 years the fraction is 0.234191, or about 23.4%. Enter 2 [ENTER] 12000 [+/-] 5730 [÷] [Y^X]. Equivalently $\lambda = \ln 2 / 5730 = 1.20968 \times 10^{-4} \text{ yr}^{-1}$ and $N/N_0 = e^{-\lambda t}$. Check: 12,000 years is just over two half-lives, so the fraction must be a little below one quarter. This is a simplified decay model. An actual dating measurement also needs its calibration and assumptions about initial activity and contamination.

4.6 When a pendulum ceases to be 'small angle'

A simple pendulum of length 0.800 m has the small-angle period $T_0 = 2\pi\sqrt{L/g} = 1.79428 \text{ s}$. At amplitude 30° (converted to $\theta_0 = \pi/6$ radians), the first finite-amplitude correction is $T \approx T_0 \left[1 + \theta_0^2/16 \right] = 1.82503 \text{ s}$. First calculate and store T_0 ; then enter 30 [ENTER] 180 [÷] f PI [×] f X² 16 [÷] 1 [+] [RCL] 00 [×]. The correction is about 1.7%. This is an approximation from an expansion, not an exact large-angle period. A test at 5° should give a much smaller correction; if it does not, inspect degree-to-radian conversion.

5. Direct-current electronics

5.1 Choose an LED resistor with a supply tolerance

Problem. A 3.30 V supply drives an LED whose forward voltage is about 2.00 V at 5.0 mA . Choose a resistor and estimate dissipation. Assume the LED voltage at the operating current; it is not a constant over all currents.

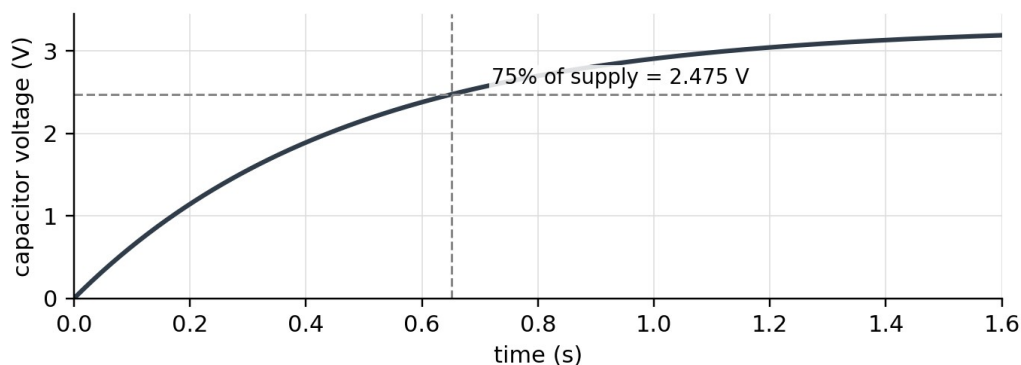
$$R = \frac{V_S - V_F}{I} = \frac{3.30 - 2.00}{0.0050} = 260\ \Omega, P_R = I^2 R = 6.5\text{ mW}.$$

On the DM42. $3.3\text{ [ENTER] } 2\text{ [-] } 0.005\text{ [÷]}$ returns $260\ \Omega$. A standard $270\ \Omega$ part gives an estimated 4.81 mA at these nominal voltages. Type $3.3\text{ [ENTER] } 2\text{ [-] } 270\text{ [÷] } 1000\text{ [×]}$ to display current in mA. If a worst-case design matters, evaluate at maximum supply and minimum plausible forward voltage; do not calculate a single nominal current and call it a bound.

5.2 Time a capacitor crossing an ADC threshold

Problem. A $47\ \mu\text{F}$ capacitor charges through $10.0\text{ k}\Omega$ from a 3.30 V ideal source. How long until it reaches 75% of supply? Assume initially discharged capacitor and negligible ADC input current.

$$\tau = RC = 0.470\text{ s}, V_C(t) = V_S(1 - e^{-t/\tau}), t = -\tau \ln(1 - 0.75) = 0.651558\text{ s}.$$



Ideal charging curve and the 75% crossing.

On the DM42. Work with seconds. $10000\text{ [ENTER] } 47\text{ EEX } 6\text{ [+/-] [×] [STO] } 00$ saves τ . Then $1\text{ [ENTER] } 0.75\text{ [-] f LN [+/-] [RCL] } 00\text{ [×]}$ returns 0.65156 s . The inverse calculation at $t = 0.600\text{ s}$ gives $V_C = 2.37935\text{ V}$: compute $0.6\text{ [RCL] } 00\text{ [÷] [+/-] f E ↑ X [+/-] } 1\text{ [+]} 3.3\text{ [×]}$. That sequence transforms $e^{-t/\tau}$ into $1 - e^{-t/\tau}$ by changing its sign before adding 1.

Sanity check. At one time constant the capacitor reaches $1 - e^{-1} = 63.2\%$ of the supply. Seventy-five per cent must take longer than 0.470 s . A real electrolytic capacitor may have a broad tolerance; retain more calculator digits for arithmetic, then report a time consistent with component tolerances.

5.3 Estimate a loaded Thévenin source

For the divider of §1.2, its open-circuit Thévenin voltage is $V_{th} = 1.05510\text{ V}$ and resistance is $R_{th} = R_T \parallel R_B = 3.19728\text{ k}\Omega$. A $10\text{ k}\Omega$ meter sees $V_{th} R_M / (R_{th} + R_M) = 0.799485\text{ V}$. This *independent circuit representation* reproduces the previous result. Check units: a $\text{k}\Omega$ value in both numerator and

denominator cancels. Store V_{th} and R_{th} when comparing several different loads; the source circuit need not be solved anew.

5.4 Infer a time constant from two voltage readings

Suppose a step response has reached 20 % of its asymptotic value after 0.105 s, and 60 % after 0.431 s. For an ideal first-order charge, $t = -\tau \ln(1 - q)$, so $\tau = -t / \ln(1 - q)$. Use 0.105 [ENTER] 0.8 f LN [+/-] [÷] to obtain $\tau \approx 0.4706$ s; for the second reading use 0.431 [ENTER] 0.4 f LN [+/-] [÷] and obtain $\tau \approx 0.4704$ s. Their close agreement supports a single-exponential model. If they differed markedly, ADC loading, capacitor leakage, inaccurate readings, or a second time constant could be involved. The direction of division matters: time divided by the positive magnitude of a logarithm gives seconds.

6. Complex numbers and alternating current

Set MODES → DEG for phase angles in degrees, and choose {RECT} or {POLAR} in MODES for complex display. A complex number is one stack object. In rectangular mode, a [ENTER] b f COMPLEX builds $a + ib$ from real part in Y and imaginary part in X. In polar mode, the same operation treats Y as magnitude and X as angle. This mode dependence is significant: a change of display mode changes the interpretation **at the moment of COMPLEX**, although it does not change an already stored complex value. The SwissMicros guide illustrates rectangular 0.2343 i1.4456 and polar 1.4645 ∠80.7937.

6.1 A series RLC circuit below resonance

Problem. Apply $10.0 V_{rms}$ at 100 Hz to a series $R = 220 \Omega$, $L = 47 mH$ and $C = 10 \mu F$. Compute impedance and current, taking source voltage as phase reference. Ideal lumped components are assumed.

$$Z = R + i \left(\omega L - \frac{1}{\omega C} \right), \omega = 2\pi f.$$

At 100 Hz, $X_L = 29.53097 \Omega$, $X_C = 159.15494 \Omega$, and $Z = 220 - i 129.62397 \Omega$. Its magnitude is 255.34756Ω and angle is -30.50659° . Therefore

$$I = \frac{10 \angle 0^\circ}{Z} = 0.03374110 + i 0.01988025 A = 0.03916231 \angle +30.50659^\circ A.$$

On the DM42. Set RECT and DEG. Compute $X_L - X_C$ with 2 [ENTER] f PI [×] 100 [×] 0.047 [×] [STO] 00, then 2 [ENTER] f PI [×] 100 [×] 10 EEX 6 [+/-] [×] f 1/X for X_C , and [RCL] 00 f X<>Y [-] for $X_L - X_C$. To avoid any uncertainty about the final swap, save X_C separately in R01 and calculate [RCL] 00 [RCL] 01 [-]. With reactance in X, enter 220 f X<>Y f COMPLEX so Y=220 and X is the signed reactance. Store Z in R02. Then 10 [RCL] 02 [÷] gives the complex current; change to MODES → POLAR to read its magnitude and angle. To return to rectangular, select RECT. The guide documents both forms and the COMPLEX stack operation.

Check. $f_0 = (2\pi \sqrt{LC})^{-1} = 232.151 Hz$. At 100 Hz the circuit is capacitive, so current **leads** voltage. Also $I_{rms}|Z| = 10.0 V$. Capacitor voltage can exceed supply magnitude at some frequencies; phasor voltages add as complex quantities, not as scalar magnitudes.

6.2 An RC filter at a known signal frequency

Problem. A $10.0\text{ k}\Omega$ series resistor feeds a 10.0 nF capacitor to ground. Output is across the capacitor. With negligible output loading, find gain and phase at 1.00 kHz .

$$H(i\omega) = \frac{1}{1 + i\omega RC}, f_c = \frac{1}{2\pi RC} = 1591.55\text{ Hz}.$$

At 1.00 kHz , $H = 0.7169568 - i0.4504772$, $|H| = 0.8467330$, and $\arg H = -32.14191^\circ$. Form the denominator in RECT: 1 [ENTER] 0.6283185307 f COMPLEX, where $\omega RC = 2\pi(1000)(10000)(10^{-8})$. Then f 1/X gives H; switch to POLAR to read magnitude and phase. Alternatively, construct the denominator from the original values on the calculator rather than entering its evaluated imaginary part. **Check:** at f_c , gain is $1/\sqrt{2}$, so gain at 1 kHz should be greater than 0.707 .

Numerical caution. The familiar $20\log_{10}|H|$ is a voltage-gain level; it is about -1.45 dB here. A quoted phase of -32° is relative to the sinusoidal input under the $e^{i\omega t}$ convention.

6.3 Average power from an AC phasor

For the series RLC circuit in §6.1, only the resistor dissipates average power in the ideal model:

$$P = I_{rms}^2 R = (0.03916231)^2 (220) = 0.337411\text{ W}.$$

On the DM42 switch to POLAR display, recall the complex current from a stored variable or recompute $10/Z$, take its magnitude with ABS, square it, and multiply by 220 . Check by

$P = VI \cos \phi = 10(0.03916231) \cos 30.5066^\circ$. The two routes should agree. Multiplying 10 V by 39.16 mA without the power factor yields apparent power in VA, not real power in W. The sign of reactive power depends on the chosen convention; dissipated resistor power is positive.

7. Matrices and coupled systems

7.1 A three-node resistor network

Problem. Three circuit nodes have $1\text{ k}\Omega$ links 1–2 and 2–3. Node 1 also has two $1\text{ k}\Omega$ paths to ground; nodes 2 and 3 each have one. An ideal 5.00 mA source injects current into node 1 from ground. Find V_1, V_2, V_3 . Conductances are exact for this illustrative network.

Kirchhoff's current law gives $AV = b$. The entries of A are in mS and those of b in mA, so the unknowns are volts:

$$\begin{pmatrix} 3 & -1 & 0 \\ -1 & 3 & -1 \\ 0 & -1 & 2 \end{pmatrix} \begin{pmatrix} V_1 \\ V_2 \\ V_3 \end{pmatrix} = \begin{pmatrix} 5 \\ 0 \\ 0 \end{pmatrix}.$$

On the DM42. MATRIX $\rightarrow$ SIMQ, enter 3 unknowns. This creates MATA, MATB, and MATX. Select {MATA} and populate in row-major order (rows 3, -1, 0; -1, 3, -1; 0, -1, 2), moving with the matrix editor's $\{\rightarrow\}$ after each entry. Press [EXIT]. Select {MATB}; populate 5, 0, 0, then exit. Select {MATX} to solve and inspect its elements in the editor. The SwissMicros guide explicitly specifies this SIMQ workflow

and its three matrices. Values entered are **mS** and **mA**; entering siemens with milliamperes would scale the answer by 10^3 .

Results. $(V_1, V_2, V_3) = (25/13, 10/13, 5/13) V = (1.92308, 0.769231, 0.384615) V$. Substitute into the second row: $-1.92308 + 3(0.769231) - 0.384615 = 0$ mA. At the source node $3V_1 - V_2 = 5$ mA. These residual checks catch transcription mistakes in the matrix editor.

Why solve instead of invert? One linear solve directly obtains V ; explicitly forming A^{-1} adds work and can magnify numerical issues. If two source cases share the same A , keep the coefficient matrix and change b .

7.2 Rotate many vectors with one matrix

The rotation of §2.2 is a matrix product. If four measured points share 37° , construct

$Q = \begin{pmatrix} \cos 37^\circ & -\sin 37^\circ \\ \sin 37^\circ & \cos 37^\circ \end{pmatrix}$ once and multiply each two-element column vector by it. 2 [ENTER] 2 MATRIX

→ **NEW** creates a 2×2 matrix in X; **{EDIT}** enters its four elements. Save that matrix to a named variable such as **ROT37**. Construct each 2×1 column, recall **ROT37** into Y and the column into X, and multiply. In this application $Q^{-1} = Q^T$; reversing the coordinate transformation corresponds to the transpose, not a new guess at the angle. Check $Q^T Q = I$ approximately. Rounding sine and cosine to three decimals *before* storing Q will weaken that identity.

8. Numerical root finding

A solver seeks a zero of a residual $f(x)$. First write a physically meaningful equation as $f(x) = 0$ and choose an interval or guesses that isolate the wanted root. The DM42 Solver works with a short labeled function containing **MVAR** declarations, followed by an expression that evaluates the residual. That small setup serves this application; programming methods are reserved for Volume II.

8.1 When does an RC voltage reach 3.3 V?

A capacitor charges towards $10.0 V$ with $\tau = 0.470$ s. Its voltage equals $3.30 V$ when

$$f(t) = 10(1 - e^{-t/0.470}) - 3.30 = 0.$$

The analytic check is $t = -0.470 \ln(0.670) = 0.18822446$ s. To use **SOLVER**, create a new program in **PRGM** mode with the following *complete minimal function* (enter the named operations through their keys or function catalog):

```

LBL "RCT"
MVAR "T"
RCL "T"
0.47
÷
+/-
E↑X
+/-
1
+
10
×
3.3
-
END

```

Start a fresh program area with `GTO . .`, toggle `[PRGM]`, enter the lines in order, then toggle `[PRGM]` again. In `SOLVER`, select `RCT`; enter an initial guess such as `0.2` into the `{T}` variable-menu key. With no number being entered, press `{T}` to solve. The result is `0.18822446 s`. Confirm by calculating $10(1 - e^{-t/0.47})$ separately: it returns `3.30 V` to displayed precision. A solution $t < 0$ would be mathematically calculable for some altered targets yet physically outside a charge started at $t = 0$.

Numerical caution. Close to $t = 0$, evaluating $1 - e^{-t/\tau}$ subtracts two nearly equal numbers. If the target is extremely small, solving with $e^{-t/\tau} - 1$ using the documented `E↑X-1` function or solving the analytic logarithmic expression can improve accuracy. If the model can be inverted exactly, the Solver is best used to confirm or to prepare for a more complicated model.

8.2 A root with no elementary closed form

Consider $e^{-x} = x$, which appears after nondimensionalisation in some transient models. The root lies between 0 and 1 because $e^0 - 0 = 1$ and $e^{-1} - 1 < 0$. Replace the residual body of the previous example with `RCL "X"`; `+/-`; `E↑X`; `RCL "X"`; `-`, and declare `MVAR "X"` after a new global label. From a guess near 0.5, `SOLVER` returns $x = 0.5671432904$. Independently, both e^{-x} and x should agree to the shown digits. A near-zero residual is necessary but does not by itself prove you found the intended root of a model with multiple solutions. Inspect your interval and the physical domain.

8.3 Avoid subtraction of nearly equal roots

The positive root of $10^{-10}x^2 + x - 1 = 0$ is algebraically $x = (-1 + \sqrt{1 + 4 \times 10^{-10}}) / (2 \times 10^{-10})$. Direct evaluation subtracts numbers close to one. Multiply numerator and denominator by the conjugate to get a stable expression:

$$x = \frac{2}{1 + \sqrt{1 + 4 \times 10^{-10}}} = 0.9999999999 \dots$$

On the DM42 evaluate the stable expression with `4 EEX 10 [+/-] 1 [+] f SQRT 1 [+] f 1/X 2 [×]`. This deliberately begins from 4×10^{-10} (enter `4 EEX 10 [+/-]`) and adds one. A decimal calculator's high precision delays cancellation, but it does not change the numerical principle. Substitute the root into the original polynomial and check its order of magnitude; rounding a near-zero residual is not proof that all printed root digits are reliable.

9. Numerical integration

9.1 Joule energy from a decaying voltage

Problem. A $1.00\text{ k}\Omega$ resistor is driven by $v(t) = 3.30 e^{-t/0.470}\text{ V}$ for $0 \leq t \leq 1.00\text{ s}$. Find the heat dissipated. Treat R as constant and the voltage function as prescribed.

$$E = \int_0^1 \frac{v(t)^2}{R} dt = \int_0^1 \frac{3.30^2}{1000} e^{-2t/0.470} dt = 2.52283934\text{ mJ}.$$

Setup on the DM42. The $\int f(x)$ application expects a labeled integrand with an MVAR for the integration variable. A compact complete integrand is:

```
LBL "RHEAT"
MVAR "T"
RCL "T"
0.47
÷
2
×
+/-
E↑X
3.3
X²
×
1000
÷
END
```

Create it in a fresh program area as in §8.1. Select $\int f(x) \rightarrow \text{RHEAT}$; choose $\{T\}$ as the integration variable, store 0 into $\{LLIM\}$, 1 into $\{ULIM\}$, and 0.000001 into $\{ACC\}$ as a starting accuracy factor, then press $\{f\}$. The SwissMicros guide states that the integral approximation is returned in X and a computational uncertainty in Y . Report **2.523 mJ** after multiplying the result in joules by 1000. Inspect the returned uncertainty; it is numerical computation uncertainty, not uncertainty in R or the voltage measurement.

Independent check. Integration by hand gives $E = 3.30^2/1000 \cdot 0.470/2 (1 - e^{-2/0.470}) = 0.00252283934\text{ J}$. Integrating beyond 1 s adds a small positive tail. Taking the infinite limit gives 2.55915 mJ ; the one-second result must be smaller. Numerical integration does not cure a discontinuous, singular, or badly scaled model automatically. Splitting an interval at a discontinuity is often preferable.

9.2 Check a variable-force work integral

If a springlike force follows $F(x) = 40x + 3x^3$ newtons with x in metres, its work from $x = 0$ to 0.20 m is

$$\int_0^{0.2} (40x + 3x^3) dx = 20(0.2)^2 + (3/4)(0.2)^4 = 0.8012\text{ J}.$$

The cubic term contributes only 0.0012 J. This is a

useful independent target for a minimal labeled integrand in the DM42 integration application. Use x in metres, not centimetres; a length conversion inside the cubic term is raised to the third power. The direct antiderivative offers a stronger check than simply rerunning the same numerical integration with tighter ACC .

9.3 Approximate an integral without a function program

For the exponential energy integrand of §9.1, the midpoint rule with ten equal 0.1 s intervals evaluates

$f(0.05), f(0.15), \dots, f(0.95)$ and multiplies their sum by 0.1 s. The result is 0.00250390 J , about 0.75 %

below the analytical $0.00252284 J$. For each midpoint compute $f(t) = 3.30^2 e^{-2t/0.470} / 1000$ watts, accumulate with STO+ 00, then multiply the stored sum by 0.1 s. Start by setting R00 to zero. This exercise exposes how quadrature works and is useful when a short tabulated dataset exists without an integrand program. Halve the step to test convergence; the difference between two approximations is an error clue, not always a rigorous bound.

10. Experimental data and regression

10.1 Measure a small resistor from current and voltage

Problem. Nine current–voltage observations of a nominal $4\ \Omega$ resistive device are shown below. Current is in amperes, voltage in volts. Fit $V = b + RI$, allowing a voltage offset, and interpret the slope.

I (A)	0.0	0.1	0.2	0.3	0.4	0.5	0.6	0.7	0.8
V (V)	0.018	0.412	0.815	1.215	1.611	2.024	2.411	2.819	3.213

Model. Least squares minimises $\sum_i [V_i - (b + RI_i)]^2$ in voltage units. The fitted line is

$V = (0.0152\text{ V}) + (4.000333\ \Omega)I$. The correlation coefficient is approximately 0.9999922. High correlation is compatible with a genuine 15 mV offset; it does **not** make the offset disappear.

On the DM42. Clear previous sums with CLEAR $\rightarrow$ CLΣ. Select STAT $\rightarrow$ {▼} $\rightarrow$ LINΣ, then STAT $\rightarrow$ CFIT $\rightarrow$ MODL $\rightarrow$ LINF for a linear model. For every row type **voltage first**, then current, then Σ+: 0.018 [ENTER] 0.0 f Σ+, then 0.412 [ENTER] 0.1 f Σ+, and so on for all nine pairs. This order is easy to reverse: the documented statistical accumulation is y [ENTER] x Σ+. Use STAT $\rightarrow$ CFIT $\rightarrow$ SLOPE, YINT, and CORR to examine the fitted parameters, or execute those named functions from CATALOG $\rightarrow$ FCN. The slope has units $V/A = \Omega$; the intercept has units V.

Check. At $I = 0.5\text{ A}$, the fit predicts 2.01537 V , while the observed value is 2.024 V ; the residual is $+0.00863\text{ V}$. The residual root-mean-square estimate $\sqrt{\sum r_i^2 / (9 - 2)} = 0.004628\text{ V}$ measures scatter about the fitted line under this model. Its interpretation as a statistical measurement uncertainty assumes suitable error behaviour and independent observations. If the voltage offset is due to the measurement instrument, shorting its inputs and correcting the zero may matter more than acquiring another decimal place in the slope.

Instrument note. The DM42's LINΣ mode uses six summation registers by default; storing unrelated values in that block while accumulating data can corrupt the fit. Save apparatus parameters in R00–R09 or named variables.

10.2 Population and sample spread answer different questions

Suppose repeated time readings in milliseconds are 4.8, 5.0, 5.1, 4.9, 5.2, 5.0. The mean is 5.0 ms, the sample standard deviation is 0.14142 ms, and the standard error of the mean is 0.05774 ms. Clear sums, enter each time as a one-variable datum with Σ+, then select STAT $\rightarrow$ MEAN and STAT $\rightarrow$ SDEV. The DM42's SDEV computes sample standard deviation; it does not automatically supply the standard error, which is $s / \sqrt{n}$ under independent identically distributed sampling. For a known *complete population* use $\sigma = \sqrt{\sum (x_i - \bar{x})^2 / n}$; for these six observations as a sample use the $n - 1$ denominator. A small standard error does not reveal calibration bias.

10.3 A weighted mean for unequal runs

A resistor was measured in three runs: 2 readings averaged $4.01\ \Omega$, 3 averaged $4.03\ \Omega$, and 1 averaged $3.99\ \Omega$. If all six individual readings carry comparable weight, the pooled average is $(2 \cdot 4.01 + 3 \cdot 4.03 + 1 \cdot 3.99) / 6 = 4.016667\ \Omega$. In the calculator, sum the products using the stack and divide by 6. The documented **WMEAN** function can also treat one member of each pair as its weight, but a direct expression makes the meaning of the weights explicit. Do not weight each run equally: that would give $4.01\ \Omega$ and disregard the unequal run sizes. If the runs have different individual variances, weighting by counts may itself be inappropriate; precision weighting uses inverse variances under a stated error model.

11. Probability and measurement uncertainty

11.1 A repeated-failure probability

Problem. Each of eight independent components has a 10% probability of failing during a test. Find the probability that three or more fail, under identical independent Bernoulli trials.

$$P(K \geq 3) = 1 - \sum_{k=0}^2 \binom{8}{k} (0.1)^k (0.9)^{8-k} = 0.03809179.$$

On the DM42. Compute the three terms and subtract their sum from one. In **PROB**, **COMB** evaluates $\binom{n}{k}$ with n in **Y** and k in **X**; for $k=2$, enter 8 [ENTER] 2 **PROB** → **COMB** to get 28. Then multiply by $0.1^2(0.9)^6$. The $k=0$ term is 0.9^8 ; the $k=1$ term is $8(0.1)(0.9)^7$. Store the running sum in **R00**, then subtract it from one. Result: **3.809%**. Check the mean count $np=0.8$; three or more failures should be uncommon. The binomial model fails if components share a common failure mechanism or have different probabilities.

11.2 Propagate input uncertainty through a range calculation

Suppose in §2.1 $v_0 = 18.0 \pm 0.15\ m\ s^{-1}$ and $\theta = 32.0^\circ \pm 0.5^\circ$, with independent *standard* uncertainties. For $R = v_0^2 \sin(2\theta) / g$, first-order propagation gives

$$u_R^2 \approx \left(\frac{2v_0 \sin 2\theta}{g} u_v \right)^2 + \left(\frac{2v_0^2 \cos 2\theta}{g} u_{\theta, rad} \right)^2.$$

Convert angular uncertainty to radians: $u_{\theta, rad} = 0.5\ \pi / 180 = 0.00872665$. Evaluate each contribution separately and combine with $\sqrt{a^2 + b^2}$, giving $u_R \approx 0.556\ m$ and $R = (29.7 \pm 0.6)\ m$. This is a model-based standard uncertainty, not an allowance for air drag. The derivative of a trigonometric function uses **radians**, even if you use **DEG** mode when evaluating $\sin 64^\circ$ and $\cos 64^\circ$; forgetting the $\pi / 180$ conversion exaggerates angular sensitivity by about 57.3.

11.3 Tolerances in the LED current

For §5.1, $I = (V_S - V_F) / R$, let $u_{V_S} = 0.05\ V$, u_{V_F} negligible for this calculation, and $R = 260 \pm 13\ \Omega$ with uncertainties treated as independent standard uncertainties. Then

$$u_I = \sqrt{\left(\frac{u_{V_S}}{R}\right)^2 + \left(\frac{V_S - V_F}{R^2} u_R\right)^2} = 0.315 \text{ m A}.$$

Thus $I \approx (5.00 \pm 0.32) \text{ m A}$ under **only the assumed input uncertainties**. Real LED forward-voltage spread is often material; omitting it from a risk estimate is a modelling decision, not a calculator capability. When a safety limit matters, calculate a worst-case bound in addition to a statistical estimate.

11.4 Positive tests and base rates

Suppose a diagnostic instrument identifies a defective item 95% of the time and incorrectly flags a good item 5% of the time. Only 2% of the incoming items are defective. The probability that a flagged item is genuinely defective is

$$P(D | +) = \frac{0.95(0.02)}{0.95(0.02) + 0.05(0.98)} = 0.279412.$$

Enter 0.95 [ENTER] 0.02 [×] [STO] 00; compute 0.05 [ENTER] 0.98 [×] [RCL] 00 [+], then [RCL] 00 ÷ X<>Y [÷]. The result is only about 28% despite the 95% sensitivity: false flags from the much larger good population dominate. The four percentages have different roles; swapping prevalence and specificity can produce a plausible-looking but wrong answer. As a count check, in 10,000 items there are 200 defective (190 flagged) and 9800 good (490 falsely flagged), so 190 of 680 flags are genuine.

12. Finance and workshop mathematics

The DM42 can calculate financial expressions, but it is not a dedicated TVM calculator. In particular, do not mistake functions from a separate calculator product for native DM42 finance menus. Enter the financial model explicitly and keep interest-rate periods consistent with payment periods.

12.1 Monthly payment on a rand loan

Problem. Borrow R250,000 at a nominal annual 11% rate, compounded monthly, with 60 equal end-of-month payments. Ignore fees and changing rates. Find the payment and first month's interest.

$$i = \frac{0.11}{12}, A = \frac{Pi}{1 - (1+i)^{-N}} = R5,435.61.$$

On the DM42. Save monthly rate in R00: 0.11 [ENTER] 12 [÷] [STO] 00. Compute the denominator: 1 [RCL] 00 [+] 60 [+/-] [Y^X] [+/-] 1 [+] gives $1 - (1+i)^{-60}$ via changing the sign of the power before adding one. Save denominator R01. Then 250000 [RCL] 00 [×] [RCL] 01 [÷] gives the monthly payment. The first month's interest is $Pi = R2,291.67$; the first principal reduction is approximately R3,143.94. Cross-check by discounting the 60-payment annuity back to R250,000. At $i = 0$, the formula has a removable singularity; use P/N instead.

Rounding caution. Rounding every intermediate rate or denominator to four display digits can shift the payment. Keep full precision internally and round the payment to cents only at the end. A lender's actual schedule can differ because of daily interest, fees, and payment-date rules.

12.2 Growth and effective rates

A R15,000 deposit growing at a nominal 8% compounded monthly for eight years becomes

$$F = 15000 \left(1 + \frac{0.08}{12} \right)^{96} = R\,28\,386.86.$$

On the DM42 enter 1 [ENTER] 0.08 [ENTER] 12 [÷] [+] 96 [Y^X] 15000 [×]. The effective annual rate is $(1 + 0.08/12)^{12} - 1 = 0.08299951$, or **8.30%**. 0.08 and 8% are different numbers in the expression. These amounts are nominal money values: if inflation matters, divide by the corresponding price-growth factor before interpreting purchasing power.

12.3 One CNC step, one drawing unit

A three-axis router calibrated at 400 full-equivalent steps per millimetre moves $1/400 = 0.0025\text{ mm}$ per step in that nominal coordinate system. A 0.50 mm commanded move is 200 steps. This arithmetic gives command resolution, not positioning accuracy: backlash, flexure, microstepping nonlinearity and calibration error can dominate. If a measured 200 mm command actually travels 199.60 mm , the new steps/mm calibration is $400(200/199.60) = 400.8016$. Measure travel over a long, repeatable interval before changing machine settings; a single short measurement has excessive relative error.

12.4 A break-even batch size

A custom machining setup costs R4,800. Each unit sells for R120 and incurs R42 of variable material and machining cost. Ignoring tax, returns, and capacity limits, contribution is R78 per unit. Break-even occurs when $78N - 4800 \geq 0$, so the smallest whole number is $N = 62$ units. Enter 4800 [ENTER] 120 [ENTER] 42 [-] [÷] for 61.538... and round **up** to the next integer because selling a fraction of a unit is impossible. At 61 units the net is $61(78) - 4800 = -R\,42$; at 62 it is $+R\,36$. If the quoted R42 excludes labour or packaging, update the model before using the threshold.

13. Mixed applications and numerical judgment

13.1 Estimate a derivative from a position model

Problem. A toy motion model has position $s(t) = 5t^2 e^{-t/4}$ metres, with t in seconds. Find velocity at $t = 2.00\text{ s}$ using a numerical calculation, then compare with the exact derivative. The function is smooth near $t = 2$; a finite difference makes sense here.

The central difference uses points on both sides:

$$v(2) \approx \frac{s(2+h) - s(2-h)}{2h}.$$

At $h = 0.10\text{ s}$, it gives $9.0938526\text{ m s}^{-1}$; at $h = 0.010\text{ s}$, it gives $9.0979188\text{ m s}^{-1}$; at $h = 0.001\text{ s}$, it gives $9.0979595\text{ m s}^{-1}$. Differentiation by hand gives $v(t) = 5e^{-t/4}(2t - t^2/4)$ and $v(2) = 15e^{-1/2} = 9.0979599\text{ m s}^{-1}$.

On the DM42. For each h , compute and store $s(2+h)$ in R00 and $s(2-h)$ in R01. To evaluate $s(t)$ from any typed t , store t temporarily in R02, then 5 [ENTER] [RCL] 02 f X² [×] [RCL] 02 4 [÷] [+/-] f E ↑ X [×]. Next calculate [RCL] 00 [RCL] 01 [-] 2 [ENTER] h [×] [÷], where **h** means typing the current chosen number. Check with the analytic derivative. Repeated halving at first helps because the truncation error of a

smooth central difference is proportional to h^2 . Making h arbitrarily tiny eventually magnifies subtraction and input noise; for experimental samples, choose a step large enough to rise above measurement resolution.

13.2 A capacitor for a brief current pulse

Problem. A wireless module draws an extra 120 mA for 0.40 s . How much capacitance alone would be required to limit an ideal supply droop to 0.50 V ? Under constant-current and ideal-capacitor assumptions, $I = C \Delta V / \Delta t$, so

$$C = \frac{I \Delta t}{\Delta V} = \frac{0.120(0.40)}{0.50} = 0.096\text{ F} = 96\text{ mF}.$$

On the DM42 enter 0.120 [ENTER] 0.40 [$\times$] 0.50 [$\div$]. The result is much larger than a typical small decoupling capacitor. A 24 mF capacitor would ideally permit a 2.0 V drop over the same pulse. This simple charge-budget calculation can quickly rule out a design idea. ESR produces an additional instantaneous drop $I R_{\text{ESR}}$, and the source and regulator may supply part of the transient current; measure the real current waveform before committing to a part.

13.3 Refraction and a critical-angle boundary

A ray passes from glass with refractive index $n_1 = 1.50$ into air with $n_2 = 1.00$, measured from the surface normal. At incident angle 30° , Snell's law gives $\sin \theta_2 = (n_1 / n_2) \sin 30^\circ = 0.75$, so $\theta_2 = 48.5904^\circ$. With DEG mode selected, enter 1.5 [ENTER] 30 f SIN [$\times$] f ASIN. The critical angle for total internal reflection is $\theta_c = \arcsin(n_2 / n_1) = 41.8103^\circ$; enter 1 [ENTER] 1.5 [$\div$] f ASIN. At incidence above θ_c , attempting to take the arcsine of the formal refracted sine greater than one signals that **no transmitted propagating ray** exists under the simple ray model. It is a physical boundary, not a request to switch the calculator into complex-results mode. Ensure both angles are measured from the normal, not from the surface.

13.4 Estimate a resonant-frequency tolerance

The series $L = 47\text{ mH}$ and $C = 10\text{ }\mu\text{F}$ of §6.1 resonate ideally near 232.15 Hz . If L and C independently range from 95% to 105% of nominal, the smallest possible ideal frequency occurs at *both* values 105%:

$f_{\min} = 232.151 / 1.05 = 221.097\text{ Hz}$. The largest occurs at both values 95%:

$f_{\max} = 232.151 / 0.95 = 244.370\text{ Hz}$. Type the nominal frequency once, save it, and divide by 1.05 and 0.95.

The shortcut works here because $f_0 \propto (LC)^{-1/2}$ and both components are scaled by the same factor in the extrema. Real inductors have winding resistance and frequency-dependent properties; this interval is a component-tolerance estimate for an ideal model, not a promise about the measured peak response.

13.5 Days between two dates

The SwissMicros function list includes YMD and DDAYS. Select f YMD so that numeric dates use YYYY.MMDD. To count days from 23 September 2026 to 31 December 2026, enter 2026.0923 [ENTER] 2026.1231 f DDAYS: X is **99 days**, because the earlier date is in Y and the later date in X. The fraction after the decimal point encodes month and day; it is **not** a fraction of a year, and ordinary subtraction of the two encoded numbers is wrong. Use DATE+ with a date in Y and a day count in X for forward date arithmetic. Write down the active DMY/MDY/YMD setting when reproducing a date calculation from someone else's calculator.

13.6 Inspect a protocol field in another base

A 16-bit configuration word has decimal value 858, or hexadecimal 35A (often written 0x035A in software), or binary 0000001101011010. Set **f DECM**; enter 858; execute **f HEXM** to inspect the hexadecimal value and **f BINM** for the binary interpretation. The SwissMicros guide notes a default 36-bit word size for base operations, adjustable with the extended **WSIZE** function from 1 to 64 bits. To reason specifically about a 16-bit field, return to **f DECM**, execute **16 f WSIZE**, then re-enter the decimal value 858 before selecting another base. A word-size change can affect sign interpretation and overflow. The bit at position zero is the least significant bit. A displayed binary field's padding zeros are part of its chosen representation, not evidence that the underlying calculator stores this number in a 16-bit scalar.

Addendum. DM42 for Programmers

Hexadecimal, binary, bit manipulation and machine arithmetic

The DM42 is not a microprocessor emulator, but it is a useful bench companion when the question is, “What exact bit pattern is this code producing?” This addendum assumes you already understand ordinary source code and want a disciplined way to inspect widths, encodings, masks, register fields, packet bytes and overflow. The examples use C notation for intent and DM42 operations for independent verification. They are deliberately short enough to repeat beside a debugger or data sheet.

Firmware boundary. The current SwissMicros guide identifies Free42 3.2.8 extensions. That build provides configurable WSIZE, signed/unsigned interpretation, wrapped/non-wrapped arithmetic, the four base modes, AND, OR, XOR, NOT, BIT?, and the classic ROTXY. Free42 3.3 later added native shifts, a Carry flag interface, set/clear-bit commands, population count, masks and IEEE-format converters. Those later commands are **not** used here as DM42 instructions. Where the task needs one of them, the method is built from functions verified in the current DM42 function list. This distinction matters: a function visible in a recent desktop Free42 manual is not automatically present on a DM42.

A working contract for every bit calculation

Before touching the calculator, write four items at the top of the page:

1. **Width:** for example 8, 12, 16, 32 or 64 bits.
2. **Interpretation:** signed two’s complement or unsigned.
3. **Overflow policy:** reject, clamp, or wrap modulo 2^w .
4. **Bit order:** bit 0 is the least significant bit; byte order is a separate issue.

Those items are part of the value’s type. Hexadecimal text alone does not supply them. FF may mean 255, -1, an opaque byte, two packed nibbles, or a character code. The calculator cannot infer which contract your firmware, protocol or file format intended.

The most reliable DM42 setup begins with `f BRESET`, which restores 36-bit, signed, non-wrapped BASE behavior. Then enter the intended width in ordinary decimal and execute `f WSIZE`; for an 8-bit job use `8 f WSIZE`. Check it at any time with `f WSIZE?`. `BSIGNED` and `BWRAP` are toggles, not setters. A dot beside `BSGN` or `BWRP` in the BASE MODES page shows that mode is active; flags 78 and 79 also report the states. Do not toggle blindly. Inspect the dots, or reset and configure from a known baseline.

P.1 Machine words: width is part of the answer

A word of width w has exactly 2^w possible bit patterns. Unsigned interpretation assigns the values 0 through $2^w - 1$. Two's-complement signed interpretation assigns -2^{w-1} through $2^{w-1} - 1$. Half of the patterns are non-negative in either interpretation; the patterns with the top bit set change meaning.

Width	Hex digits	Unsigned range	Signed two's-complement range
8	2	0 ... 255	-128 ... 127
12	3	0 ... 4095	-2048 ... 2047
16	4	0 ... 65,535	-32,768 ... 32,767
32	8	0 ... 4,294,967,295	-2,147,483,648 ... 2,147,483,647
64	16	0 ... 18,446,744,073,709,551,615	-9,223,372,036,854,775,808 ... 9,223,372,036,854,775,807

The bits do not change; the interpretation does.

7	6	5	4	3	2	1	0
1	1	1	1	1	0	1	1

unsigned: 251

signed two's complement: $251 - 256 = -5$

One byte interpreted two different ways.

For a w -bit pattern u , the signed value is

$$s = \begin{cases} u, & u < 2^{w-1} \\ u - 2^w, & u \geq 2^{w-1} \end{cases}$$

Thus the byte FB is 251 unsigned and $251 - 256 = -5$ signed. No bit changes when the interpretation changes. On the DM42, configure 8 WSIZE, select unsigned mode, enter FB in hexadecimal, and note 251 in decimal. Toggle to signed mode and the same eight-bit pattern represents -5. This is a view change, not a negation.

Why WSIZE matters. NOT 00 is meaningless without a width. In 8 bits it produces FF; in 16 bits it produces FFFF. Likewise, the sign bit is bit 7 for an 8-bit word and bit 31 for a 32-bit word. The SwissMicros guide allows WSIZE from 1 through 64 and defines the most significant bit at the selected width as the sign bit. A pasted diagnostic such as 0x8000 should therefore be inspected as 16 bits if it came from an int16_t, not left at the 36-bit default.

C parallel. Fixed-width types make the same contract explicit:

```
uint8_t u = 0xFB; /* 251 */
int8_t s; /* desired interpretation: -5 */
memcpy(&s, &u, 1); /* copy representation without a value conversion */
```

Converting an out-of-range unsigned value directly to a signed type can be implementation-defined in C; copying the representation and checking the target implementation is the careful systems-programming choice. On the calculator, the signed-mode toggle is explicitly a two's-complement reinterpretation.

P.2 Base modes and entering real data-sheet values

The BASE application displays real integer values in hexadecimal, decimal, octal or binary. Use {BASE} → {HEXM}, {DECM}, {OCTM}, or {BINM}; the same functions can be called by name. In hexadecimal mode, {A...F} both selects HEXM and presents the six letter digits. The four arithmetic keys are temporarily remapped while the BASE menu is open: +, −, ×, ÷, and +/- become integer BASE+, BASE−, BASE×, BASE÷, and two’s-complement BASE+/-.

Convert 3735928559. Reset, select 32 WSIZE, unsigned and non-wrapped. In decimal base enter 3735928559, then select hex: the pattern is DEADBEEF. Select binary to see 1101111010101101101111011101111. In signed mode that same 32-bit pattern is −559,038,737. This familiar “dead beef” value is useful because every byte is visually distinct: DE AD BE EF.

Convert a hex literal from C. Select {A...F} and enter C0A80164. Switch to decimal to obtain 3,232,235,876. For most protocol work, however, the packed bytes are more useful than that decimal integer; §P.6 extracts them.

Binary display and SHOW. Holding SHOW displays all bits when the word is 44 bits or shorter; for a wider binary word the guide reports it in hexadecimal. This is a display constraint, not loss of bits. A 64-bit value is normally easier to audit as sixteen hex digits, grouped on paper into four- or eight-digit units.

Leading zeroes. A numeric display may omit leading zeroes, but a register field does not. Record the declared width. 35A as a 16-bit word is 035A; as a 12-bit word it is exactly 35A. The difference affects the top-bit position and therefore signedness.

Octal has a role. Octal groups bits in threes. It remains convenient for Unix permission masks: decimal 493 is octal 755 and binary 111 101 101. With 9-bit unsigned mode, inspect 755 in octal, then binary. The groups read owner 111, group 101, other 101, corresponding to *rwX r-X r-X*. Do not type the C prefix 0, 0x, or 0b as part of the calculator’s digits; choose the base mode instead.

P.3 Signed, unsigned and wrapped arithmetic

The current DM42 BASE extensions separate interpretation from overflow policy:

Setting	Meaning	Typical use
Signed	Top bit has weight -2^{w-1}	int8_t, signed samples, deltas
Unsigned	Every bit has positive place value	addresses, masks, counters, byte fields
Non-wrapped	Reject out-of-range inputs/results	checking whether a result fits
Wrapped	Discard bits above bit $w - 1$	modelling modular machine arithmetic

In non-wrapped mode, an argument outside the selected range produces **Invalid Data**; an out-of-range result produces **Out of Range**. If flag 24, Range Error Ignore, is set, the non-wrapped result is clamped to the nearest representable value. Clear flag 24 for overflow investigations: saturation can hide the exact event you meant to detect. In wrapped mode, arguments and results are reduced to the selected width by discarding higher bits.

Unsigned wrap: an 8-bit timer

An 8-bit free-running timer contains 250 and advances by 10. In C, conversion to `uint8_t` reduces modulo 256:

```
uint8_t next = (uint8_t)(250u + 10u); /* 4 */
```

On the DM42, reset, choose 8 WSIZE, unsigned and wrapped. In decimal BASE mode calculate 250 ENTER 10 +; X is 4. In hex, FA + 0A = 04. The full mathematical result is 260, the retained byte is $260 \bmod 256 = 4$, and the carry-out is $\lfloor 260 / 256 \rfloor = 1$.

Turn wrapped mode off and repeat. The same addition reports an out-of-range result instead of silently keeping 04. This is a good two-pass audit: non-wrapped mode answers “does it fit?”, while wrapped mode answers “which bits would remain?”

Signed wrap: the top bit is not a magnitude bit

With 8-bit signed wrapped arithmetic, $127 + 1$ gives the pattern 80, interpreted as -128 . There is no carry out of bit 7: $7F + 01 = 80$. Nevertheless signed overflow occurred because two positive operands produced a negative result. Carry and signed overflow are different predicates.

For two’s-complement addition of w -bit values a , b , and result r , signed overflow occurs when the operands have the same sign and the result has a different sign:

$$V = ((a \oplus r) \& (b \oplus r) \& 2^{w-1}) \neq 0.$$

The equivalent source expression uses unsigned representations to avoid undefined signed overflow:

```
uint8_t ua = 0x7F, ub = 0x01;
uint8_t ur = (uint8_t)(ua + ub);
bool overflow = ((ua ^ ur) & (ub ^ ur) & 0x80u) != 0;
```

On the DM42, calculate the wrapped result, then use XOR, AND, and sign mask 80 to reproduce the test. The calculator’s non-wrapped signed mode is simpler when all you need is a fit check.

BASE+/- is width-sensitive negation

Two’s-complement negation is $\text{NOT } x + 1$, reduced to the word width. In 8 bits, negating 01 produces FF (-1 signed, 255 unsigned). The most-negative signed value is its own bit-pattern result under wrapping: negating 80 yields 80. Mathematically, $+128$ cannot be represented in signed 8-bit form. That asymmetry is why `abs(INT_MIN)` is troublesome in C and why a non-wrapped calculator should report overflow for the same operation.

P.4 Logic, masks and individual bits

Bitwise logic operates column by column:

Operation	Result bit is 1 when...	Common purpose
AND	both input bits are 1	keep selected fields
OR	either input bit is 1	set flags or combine non-overlapping fields
XOR	input bits differ	toggle flags, parity/checksum accumulation
NOT	input bit is 0	invert within the selected word width

The DM42 uses the usual RPN binary order: put the first operand in Y and the second in X, then execute the logical function. AND, OR and XOR are commutative, so order does not affect their value, but writing a consistent value ENTER mask AND pattern makes the intent easier to review.

Keep one byte

To extract the low byte of DEADBEEF, use a 32-bit unsigned word and calculate:

```
uint32_t low = 0xDEADBEEFu & 0xFFu; /* 0xEF */
```

On the DM42 in hexadecimal mode: DEADBEEF ENTER FF f AND gives EF. A low- n -bit mask is $2^n - 1$; for 12 bits it is FFF, and for 5 bits it is 1F. You can build a mask in decimal as 2 ENTER n Y^X 1 -, then return to BASE mode, but typing the corresponding hex mask is usually clearer during an audit.

Set, clear and toggle one bit

For bit number k , define $m = 1 \ll k$. Then:

```
x |= m;          /* set */
x &= ~m;         /* clear */
x ^= m;          /* toggle */
bool b = (x & m) != 0;
```

On the current DM42 there are no native SB or CB commands. Build or type the mask and use OR, NOT+AND, or XOR. With an 8-bit word, value 25 and bit 3 mask 08:

- 25 OR 08 → 2D, setting bit 3.
- 25 AND NOT 08 → 25, because that bit was already clear. Execute NOT on 08 at **8-bit width** to obtain F7.
- 25 XOR 08 → 2D, toggling bit 3.

BIT? is convenient for a direct test. Put the value in Y and the bit index in X, then execute BIT?; bit positions count from zero at the right. Interactive use displays Yes or No; in a program it conditionally executes or skips the following line. For 0x25, 25 ENTER 5 BIT? tests bit 5 and answers Yes. The command is a test, not an extractor.

Write masks in field form

A field occupying bits h down to l has width $n = h - l + 1$ and mask

$$M = ((1 \ll n) - 1) \ll l.$$

For bits 7:5, $n = 3$, so the mask is binary 11100000 or hex E0. Extraction is $(x \& M) \gg l$; insertion of already range-checked value v is $(x \& \sim M) | ((v \ll l) \& M)$. Clearing before setting prevents stale one-bits from surviving.

P.5 Shifts, rotates and the carry boundary



The difference between a logical left shift and a left rotate.

A logical shift discards bits at one end and introduces zeroes at the other. An arithmetic right shift copies the sign bit into the new high positions. A rotate reintroduces the discarded bits at the opposite end. These are three different operations even when they agree for a particular value.

Current-firmware limitation. The DM42 build documented for this book does not include Free42 3.3's SL, SR, ASR, rotate-through-carry commands, or SC/CC/C?. It does include classic ROTXY, which rotates the 36-bit value in Y by the count in X: a positive count rotates right and a negative count rotates left. It is a **36-bit** operation in the current SwissMicros function list, independent of a smaller protocol field you may be studying. Use it directly for 36-bit work; for 8-, 16- or 32-bit rotations, use the width-specific formula below.

Reconstruct shifts with arithmetic

For unsigned w -bit x and $0 \leq n < w$:

$$LSL_n(x) = (x 2^n) \bmod 2^w, LSR_n(x) = \lfloor \frac{x}{2^n} \rfloor.$$

Set unsigned mode. A wrapped BASE multiplication by 2^n reproduces a left shift; an integer BASE division by 2^n reproduces a logical right shift for a non-negative unsigned value. Example at 8 bits: $96 \ll 1$ is 2C, because hexadecimal $0x96 \times 2 = 0x12C$ and the high 1 is discarded. $96 \gg 1$ is 4B.

An arithmetic right shift of a negative two's-complement value usually rounds toward negative infinity. Thus -5 arithmetic-shifted right once is -3 , not -2 . Integer division rules differ among languages and machines, so do not substitute division without checking. A calculator reconstruction is $\lfloor x / 2^n \rfloor$; if the available integer division truncates toward zero, use $(x - (2^n - 1)) / 2^n$ for negative x before truncation.

Rotate within the declared width

For $0 < n < w$, a right rotate is

$$ROTR_w(x, n) = (x \gg n) | ((x \ll (w - n)) \& (2^w - 1)).$$

With $x=96$ hex and $w=8$, right rotation by one produces 4B; left rotation by one produces 2D. The left shift also produced 2C; the rotate differs only because old bit 7 re-enters at bit 0. Work each part separately in

unsigned mode and combine with OR. If $n=0$, return x directly; shifting by the full width is undefined in C even though a rotate definition commonly reduces the count modulo width.

```
uint8_t rotr8(uint8_t x, unsigned n) {
    n &= 7u;
    return n ? (uint8_t)((x >> n) | (x << (8u - n))) : x;
}
```

Carry-out versus retained result

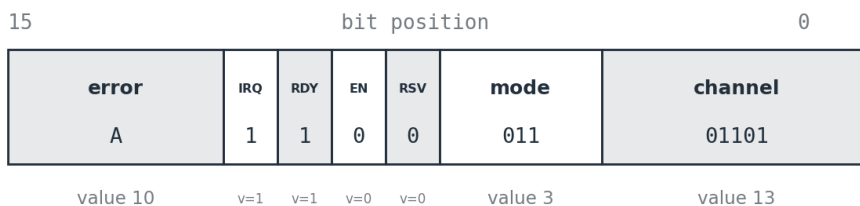
For unsigned addition, the carry-out is $\lfloor (a+b)/2^w \rfloor$ and the retained result is $(a+b) \bmod 2^w$. For unsigned subtraction, a borrow occurs when $a < b$. A CPU carry flag is a one-bit side result; it is not stored in the destination word. The current DM42 firmware does not expose the newer Free42 Carry interface, so compute it explicitly: evaluate the full sum in ordinary decimal arithmetic, compare with 2^w , then separately reproduce the wrapped word in BASE mode.

For FA + 0A at 8 bits, full sum $260 \geq 256$, carry 1, retained 04. For 7F + 01, full unsigned sum $128 < 256$, carry 0, retained 80, but signed overflow 1. This pair should be on every programmer's scratch card.

P.6 Packing, unpacking and bit-field diagrams

Packed data is easiest to review when the picture, mask and numeric check agree.

Example 16-bit status/control word: 0xAC6D



Mask, shift and interpret one field at a time; do not read the word as one decimal quantity.

A 16-bit status/control register with named fields.

Consider the 16-bit register AC6D with this illustrative layout:

Bits	Field	Extractor	Value in AC6D
15:12	error code	$(\text{word} \gg 12) \& 0xF$	10
11	IRQ pending	$(\text{word} \gg 11) \& 1$	1
10	ready	$(\text{word} \gg 10) \& 1$	1
9	enable	$(\text{word} \gg 9) \& 1$	0
8	reserved	$(\text{word} \gg 8) \& 1$	0
7:5	mode	$(\text{word} \gg 5) \& 7$	3
4:0	channel	$\text{word} \& 0x1F$	13

On the DM42, select 16-bit unsigned mode. For channel, AC6D ENTER 1F AND gives 0D. For mode, first mask with 00E0, then divide by 20 in BASE integer arithmetic; the result is 3. For the error code, mask with F000, then divide by 1000; the result is A, or 10 in decimal. Division by a power of two is a logical right shift here because the masked word is non-negative and unsigned.

To set mode to 5 while preserving all other fields, clear 00E0 and insert $5 \ll 5 = 00A0$:

```
word = (uint16_t)((word & (uint16_t)~0x00E0u) | 0x00A0u);
/* 0xAC6D becomes 0xACAD */
```

At 16-bit WSIZE, NOT 00E0 yields FF1F. Then AC6D AND FF1F gives AC0D; OR with 00A0 gives ACAD. The explicit width prevents NOT from generating an unexpectedly long complement.

Pack and unpack bytes

0xC0A80164 split into four octets

C0 byte 3	A8 byte 2	01 byte 1	64 byte 0
192	168	1	100
network-order reading: 192 . 168 . 1 . 100			

A 32-bit word split into four bytes.

The 32-bit value C0A80164 can be read as four octets C0 A8 01 64, or decimal 192.168.1.100. With word unsigned:

```
uint8_t b3 = (uint8_t)((word >> 24) & 0xFFu);
uint8_t b2 = (uint8_t)((word >> 16) & 0xFFu);
uint8_t b1 = (uint8_t)((word >> 8) & 0xFFu);
uint8_t b0 = (uint8_t)(word & 0xFFu);
```

On the DM42, extract b2 by masking 00FF0000, then integer-dividing by 10000 hex (2^{16}): the result is A8, or 168 decimal. Repeat with the appropriate mask and divisor. Pack bytes by multiplying each by its positional power and combining with OR; because the fields do not overlap, addition gives the same bits, but OR better states the intent.

For bytes 12 34 AB CD, compute

$$12 \cdot 2^{24} + 34 \cdot 2^{16} + A B \cdot 2^8 + C D = 1234 A B C D_{16}.$$

Endianness is about byte order, not bit numbering

The numeric word 0x1234ABCD is one value. In big-endian memory its increasing-address bytes are 12 34 AB CD; in little-endian memory they are CD AB 34 12. The bit-numbering convention inside each byte still runs from bit 0 at the least-significant end. A packet specification's "network byte order" normally means big-endian. The DM42 shows a numeric value, not a memory dump, so it will not reveal host endianness unless you tell it the observed byte sequence and pack those bytes explicitly.

P.7 Signedness traps and sign extension

Compare the same bits two ways

0xFFFFFFFF is 4,294,967,295 as uint32_t and -1 as int32_t. Therefore an expression such as signed_value < 1u may surprise: usual arithmetic conversions can convert the signed operand to unsigned before comparison. On a 32-bit implementation, -1 becomes 0xFFFFFFFF, and the comparison is false.

The DM42 makes the trap visible. Use 32-bit unsigned mode to see FFFFFFFF as the large positive value; toggle signed mode to see -1 . Whenever a mixed C expression looks wrong, write the converted type beside each operand before calculating its value.

Sign-extend a 12-bit sensor field

Suppose the low 12 bits of a 16-bit ADC/status word are F38. Interpreted as unsigned, F38 is 3896. Because bit 11 is set, the signed 12-bit value is $3896 - 4096 = -200$.

One robust source pattern is:

```
uint16_t raw12 = word & 0x0FFFu;
int16_t sample = (raw12 & 0x0800u)
    ? (int16_t)(raw12 - 0x1000u)
    : (int16_t)raw12;
```

On the DM42, first extract under 16-bit unsigned settings. Then change WSIZE to 12 and select signed mode to reinterpret F38 as -200 . Alternatively, stay in ordinary decimal arithmetic and subtract 4096 when the sign mask 800 is set. The latter mirrors the explicit C and avoids depending on a language's right-shift behavior for negative values.

Sign extension rule. To widen a negative w -bit two's-complement pattern, fill every newly added high bit with one. F38 at 12 bits becomes FF38 at 16 bits and FFFFFFF38 at 32 bits. Zero extension would produce 00000F38, which is $+3896$. Widening the value and widening the pattern are not interchangeable operations unless signedness is known.

Truncation discards information

Narrowing 0x1234 to 8 bits retains 0x34. In C, unsigned conversion is well-defined modulo 2^8 ; conversion to an out-of-range signed type is implementation-defined. On the DM42, 8-bit wrapped mode models the retained bit pattern, while 8-bit non-wrapped mode reveals that the original does not fit. Use both views when reviewing a cast.

P.8 Population count, positions, parity and checksums

The current DM42 does not have Free42 3.3's #B population-count command. You can still validate short values without writing a calculator program.

Count set bits by clearing the lowest one

For nonzero unsigned x , $x \& (x-1)$ clears its lowest set bit. Repeat until zero; the number of repetitions is the population count.

```
unsigned count = 0;
for (uint32_t t = x; t; t &= t - 1u)
    ++count;
```

For A53C, the sequence is $A53C \rightarrow A538 \rightarrow A530 \rightarrow A520 \rightarrow A500 \rightarrow A400 \rightarrow A000 \rightarrow 8000 \rightarrow 0000$, so the count is 8. On the DM42, use unsigned wrapped mode, keep the current value in a register, subtract 1, AND with the saved value, and count the iterations on paper. DEADBEEF contains 24 one-bits; that is a useful independent check from nibble counts $D=3, E=3, A=2, D=3, B=3, E=3, E=3, F=4$.

For one-off inspection, hexadecimal nibble weights are faster:

Hex digit	One-bits	Hex digit	One-bits
0	0	8	1
1, 2, 4	1	9, A, C	2
3, 5, 6	2	B, D, E	3
7	3	F	4

Find set-bit positions

Use **BIT?** when the range is small or the candidate position is known. To find the highest set bit of **035A**, note that the leading hex digit 3 occupies bits 11:8 and is binary **0011**; the highest one is bit 9. The lowest set bit can be found from the rightmost nonzero hex digit: **A** is **1010**, whose lowest one is nibble bit 1, hence overall bit 1.

The arithmetic identity $x \& -x$ isolates the lowest set bit in two's-complement modular arithmetic. At 16-bit width, **035A** negates to **FCA6**; **AND** gives **0002**, confirming bit 1. Use wrapped mode, because the identity relies on fixed-width negation.

XOR checksum and parity

For bytes **3A C7 10 05**, the bitwise XOR checksum is **E8**:

```
uint8_t check = 0;
check ^= 0x3A;
check ^= 0xC7;
check ^= 0x10;
check ^= 0x05;          /* 0xE8 */
```

On the DM42 at 8-bit unsigned width, combine the four bytes with repeated **XOR**. **XOR** is associative and commutative, so reordering does not change the result. Repeating the same byte twice cancels it. This checksum is good at detecting a single changed bit but weak against many structured errors; do not call it a CRC.

Parity is the population count modulo two. **E8** has three set bits, so it has odd parity. To append an even-parity bit, choose 1 so that the total number of ones becomes even. Always state whether the parity bit is included in the count and where it is stored.

Work a small CRC one bit at a time

A cyclic redundancy check is not “an XOR checksum with a better name.” It is polynomial division over bits, and the parameters are part of the format: polynomial, initial value, input/output reflection and final XOR. Two algorithms both called CRC-8 may therefore give different results.

As a reproducible example, use the bytes **C0 A8 01 64**, polynomial **07**, initial register **00**, most-significant bit first, no reflection and no final XOR. For each byte, XOR it into the 8-bit register. Then repeat eight times: if old bit 7 was one, left-shift and **XOR 07**; otherwise only left-shift. Every intermediate result is reduced to eight bits.

```
uint8_t crc = 0x00;
for (size_t i = 0; i < 4; ++i) {
    crc ^= data[i];
    for (unsigned bit = 0; bit < 8; ++bit)
        crc = (crc & 0x80u)
            ? (uint8_t)((crc << 1) ^ 0x07u)
            : (uint8_t)(crc << 1);
}
/* crc == 0x9D */
```

Input byte	Register after byte XOR	After eight update steps
C0	C0	4E
A8	E6	BC
01	BD	3A
64	5E	9D

On the DM42, use 8-bit unsigned wrapped mode. Record bit 7 with **BIT?** before each update. Model the left shift by BASE-multiplying the register by 2. If the saved top bit was set, XOR the wrapped result with **07**. For the first byte, the eight successive states are **87, 09, 12, 24, 48, 90, 27, 4E**. Completing the four rows produces **9D**.

This is intentionally a verification method, not a fast way to process a file. Its value is diagnostic: if a device reports another check byte, compare the state after each input byte. The first divergent row usually reveals whether byte order, reflection, initial value or final XOR differs. Never validate a CRC from only its width and polynomial; write the complete parameter set beside the calculation.

P.9 Overflow analysis without hand-waving

Overflow is a mismatch between a mathematical result and a destination type. A calculator display of the full mathematical result does not prove the machine result is safe, and a wrapped result does not tell you which rule was violated.

Unsigned addition and subtraction

For w -bit unsigned addition, overflow/carry occurs when $a + b \geq 2^w$. An implementation-friendly test is $r < a$, where r is the wrapped result. For subtraction $r = a - b$, underflow/borrow occurs when $a < b$.

```
uint16_t r = (uint16_t)(a + b);
bool carry = r < a;
bool borrow = a < b;
```

Check **0xFFF0 + 0x0030**: the full sum is **0x10020**; at 16 bits the retained result is **0020** and carry is 1. In non-wrapped unsigned DM42 mode, the addition is rejected. In wrapped mode it gives **20**. Ordinary decimal calculation of $65,520 + 48 = 65,568$ confirms that one bit lies beyond 65,535.

Signed addition and subtraction

Signed overflow is about sign consistency, not carry. At 8 bits:

Operation	Retained pattern	Signed result shown	Carry	Signed overflow
7F + 01	80	-128	0	1
FF + 01	00	0	1	0
80 - 01	7F	127	—	1
FE + 01	FF	-1	0	0

In standard C, overflowing a signed integer type is undefined behavior; do not rely on two's-complement wrap merely because the hardware usually does it. Perform the operation in a suitable unsigned type or a wider signed type, test the range, and convert deliberately.

Multiplication

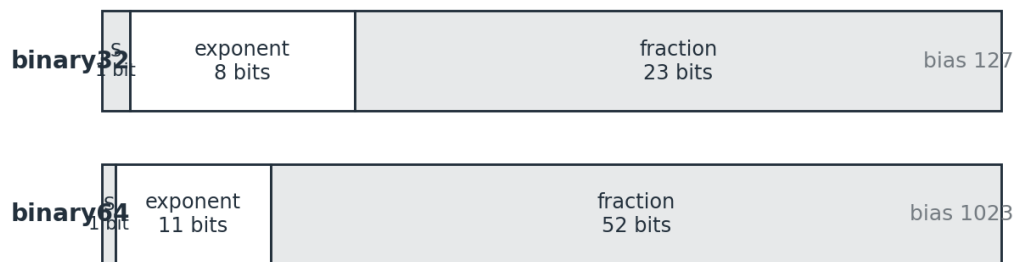
For unsigned multiplication, compute in a type at least twice as wide when available and compare with the destination maximum. Example: $300 \times 300 = 90,000$, which does not fit 16-bit unsigned; the wrapped pattern is 5F90 (24,464). On the DM42, 16-bit non-wrapped mode reports the overflow; wrapped mode confirms the retained word. For signed multiplication, compare against both the minimum and maximum because the range is asymmetric.

Saturation is not wrapping

Digital-signal code sometimes clamps $200 + 100$ to 255 in an unsigned byte; ordinary machine arithmetic wraps it to 44. The DM42's non-wrapped mode with Range Error Ignore flag 24 clamps out-of-range results, but that global flag can conceal mistakes. Prefer an explicit calculation: compute the full result, compare with the limits, and choose the endpoint intentionally. Write `sat_u8(200+100)=255`, not merely "overflow handled."

P.10 IEEE-754 binary layouts and hex conversions

IEEE 754 binary interchange layouts



normal finite value: $(-1)^s \times (1 + \text{fraction}/2^p) \times 2^{(\text{exponent}-\text{bias})}$

Field layout of IEEE binary32 and binary64 formats.

Binary floating-point hex is a representation problem, not a BASE-mode integer problem. IEEE 754 binary32 has 1 sign bit, 8 exponent bits with bias 127, and 23 stored fraction bits. Binary64 has 1, 11 and 52 bits with bias 1023. For a normal finite value,

$$x = (-1)^s \left(1 + \frac{F}{2^p} \right) 2^{E-\text{bias}},$$

where $p = 23$ or 52 . Exponent field zero denotes zeros/subnormals; an all-ones exponent denotes infinities or NaNs. The payload and quiet/signalling details of NaNs require the platform's convention and should not be guessed from a decimal printout.

Decode binary32 C1540000

Write it in binary by nibbles:

C1540000 = 1 | 10000010 | 101010000000000000000000.

- Sign $s = 1$, so the value is negative.
- Exponent field 10000010 is 130; unbiased exponent $e = 130 - 127 = 3$.
- Fraction bits encode $F / 2^{23} = 1/2 + 1/8 + 1/32 = 0.65625$.

- Significand is 1.65625; value is $-1.65625 \times 2^3 = -13.25$.

On the DM42, use 32-bit unsigned mode to isolate fields: sign mask 80000000; exponent mask 7F800000, then divide by 800000 (2^{23}); fraction mask 007FFFFF. The resulting integers can be inserted into the formula in ordinary decimal mode.

Encode binary32 3.1415927

Normalize the binary value as approximately 1.57079635×2^1 . The exponent field is $1 + 127 = 128 = 0 \times 80$. Rounding the fractional part to 23 stored bits gives 0x490FDB. Combining sign 0, exponent 0x40000000, and fraction yields **0x40490FDB**. Decoding that bit pattern gives approximately 3.1415927410125732, the nearest binary32 value, not the decimal input exactly.

The DM42 can audit the field arithmetic, but the current firmware does not provide Free42 3.3's N→BS converter. Use a language runtime, compiler, protocol library or known test vector to obtain the correctly rounded encoding, then use the DM42 to inspect its fields. Do not promise correctly rounded decimal-to-binary conversion by hand for arbitrary inputs.

Decode binary64 3FB999999999999A

This is the standard binary64 representation nearest decimal 0.1. Sign is zero. The top 12 bits are 3FB: after removing the sign, the exponent field is 0x3FB=1019, so $e = 1019 - 1023 = -4$. The fraction is a repeating 1001... pattern rounded at 52 bits. The stored value is approximately

$$0.10000000000000000055511151231257827,$$

which explains why repeated binary floating arithmetic can expose a tiny decimal residue. Other useful checks are $42.5 \rightarrow \mathbf{4045400000000000}$ and π as binary64 $\rightarrow \mathbf{400921FB54442D18}$.

The DM42 can hold a 64-bit unsigned pattern in BASE mode and can mask the 11-bit exponent with 7FF0000000000000, dividing by $2^{52} = 10000000000000$ hex. Be precise about mode: a binary64 payload is an integer bit pattern during field extraction, not a DM42 real number being “converted” merely by selecting hexadecimal display.

DM42 internal arithmetic is different

The DM42's Free42 Decimal core uses IEEE decimal128-style internal arithmetic: 34 decimal digits and a decimal exponent range. Therefore decimal 0.1 is representable exactly in its normal calculations. That makes the calculator excellent for reference decimal arithmetic, but it is **not a drop-in emulator for binary32 or binary64 rounding, subnormals, NaN payloads, fused operations or language-specific evaluation rules**. To reproduce a target's binary result, work from that target's encoded bits or a conforming software reference.

P.11 Characters, bytes and code points

The current DM42 function list includes C→N and N→C, which convert between a character and its numeric code in the calculator's character system. They are useful for ASCII-compatible characters: N→C applied to 65 produces A, and C→N applied to A returns 65. In hex, 65 is 41; lowercase a is decimal 97, hex 61.

Character	Decimal	Hex	Binary byte
0	48	30	00110000
9	57	39	00111001
A	65	41	01000001
Z	90	5A	01011010
a	97	61	01100001
LF	10	0A	00001010

Do not generalise this into “one character equals one byte.” ASCII is a 7-bit code and UTF-8 is a variable-length encoding of Unicode code points. For example, é is code point U+00E9 but UTF-8 bytes C3 A9. A DM42 glyph, its internal calculator code, its UTF-8 bytes in a file, and a Unicode code point are different layers. Use the protocol or file-format specification to choose the layer before converting.

Inspect a printable packet field

Suppose a four-byte tag is 44 4D 34 32. Interpreting each byte as ASCII gives DM42. On the DM42, convert hex 44, 4D, 34, 32 to decimal 68, 77, 52, 50 and then use N→C one at a time. Conversely, take each known ASCII-compatible character through C→N and inspect its hex byte. For control bytes such as 00, 0A or 1B, record the numeric code; absence of a printable glyph is expected.

P.12 Six mini challenges

Try each problem with a written width and mode before reading the check.

Challenge 1: decode a status byte

An 8-bit status byte is B6. Bits 7:6 are severity, bit 5 is alarm, bits 4:2 are source, and bits 1:0 are count. Extract every field.

Check. B6 = 10 1 101 10: severity 2, alarm 1, source 5, count 2. Masks are C0, 20, 1C, 03; divide masked multi-bit fields by their low-bit weight.

Challenge 2: insert a mode field safely

A 16-bit control word starts as A55A. Replace bits 7:5 with mode 3, preserving all other bits.

Check. Clear mask E0: A55A & FF1F = A51A. Insert 3<<5 = 60. Result **A57A**. Direct OR without clearing would fail whenever old field bits conflict with the new value.

Challenge 3: signed sensor value

The low 12 bits of AF38 are a signed two’s-complement sensor reading. Find the decimal reading and its 16-bit sign extension.

Check. Raw field F38; bit 11 is set; $3896 - 4096 = -200$. Sign-extended pattern **FF38**.

Challenge 4: byte order

A four-byte little-endian memory dump is 78 56 34 12. What 32-bit numeric word does it represent? How would the same value appear in network byte order?

Check. Numeric word **12345678**; big-endian/network bytes 12 34 56 78. Do not reverse bits within each byte.

Challenge 5: overflow classifiers

At 8 bits, classify $F0 + 30$ as unsigned carry and as signed overflow. Give the retained pattern and both interpretations.

Check. Full unsigned sum 120 hex; retained **20**, carry 1. Operands are -16 and $+48$ in signed interpretation, so their signs differ and signed overflow is 0; retained signed result is $+32$.

Challenge 6: IEEE field check

Decode binary32 3E200000.

Check. Sign 0; exponent $7C = 124$, so $e = -3$; fraction field begins 010..., so significand 1.25. Value $1.25 \times 2^{-3} = 0.15625$.

P.13 Base-mode warning and quick reference

The classic surprise: division became integer division

While the BASE menu is active, the physical arithmetic keys are remapped. Thus **7 ENTER 2 ÷** evaluates integer **BASE÷** and produces 3. Exit the BASE menu; decimal mode is selected automatically and the ordinary division key again gives 3.5. If a scientific calculation suddenly loses its fractional part, look for the active BASE menu before investigating the formula.

Likewise, **+/-** inside BASE is fixed-width two's-complement negation, not merely the ordinary real-number sign change. Its result depends on **WSIZE**, signedness and wrap behavior. Leave BASE before continuing an engineering calculation that expects real arithmetic.

Programmer's quick reference

Task	C expression	DM42 method, current firmware
Establish a known state	—	BRESET; set decimal width with WSIZE ; inspect BSGN/BWRP dots
Convert display base	—	HEXM , DECM , OCTM , BINM ; A...F for hex digits
Range for w bits	unsigned $0 \dots 2^w - 1$; signed $-2^{w-1} \dots 2^{w-1} - 1$	Configure width and signedness; use non-wrapped mode to test fit
Model wrap	cast to unsigned fixed width	Enable BWRAP ; arithmetic keys in BASE are integer operations
Keep selected bits	$x \& \text{mask}$	value ENTER mask AND
Set bits	$x \mid \text{mask}$	OR
Clear bits	$x \& \sim \text{mask}$	NOT mask at correct width, then AND
Toggle bits	$x \wedge \text{mask}$	XOR
Test bit k	$(x \& (1 \ll k)) \neq 0$	value ENTER k BIT?
Logical left shift	$(x \ll n) \& \text{mask}$	unsigned wrapped BASE multiply by 2^n
Logical right shift	$x \gg n$ for unsigned x	unsigned BASE integer divide by 2^n

Task	C expression	DM42 method, current firmware
Rotate a 36-bit word	implementation helper	value ENTER count ROTXY; positive right, negative left
Rotate another width	combine two shifts with OR	evaluate width-specific rotate formula
Low- n mask	$(1u < n) - 1u$	type hex mask or calculate $2^n - 1$
Extract field $h:l$	$(x >> l) \& ((1u < (h-l+1)) - 1)$	AND with positioned mask; integer-divide by 2^l
Sign-extend w bits	explicit top-bit test	if sign bit set, subtract 2^w ; or reinterpret at WSIZE w signed
Population count	clear-lowest-one loop	repeat x AND $(x-1)$ and count
Byte-wise checksum	repeated ^	repeated XOR at 8-bit width
CRC step	conditional polynomial XOR	save top bit; wrapped $\times 2$; conditionally XOR polynomial
Character code	encoding-specific	$C \rightarrow N$, $N \rightarrow C$; use ASCII-compatible range cautiously
IEEE hex fields	masks and shifts	treat payload as unsigned bits; isolate sign/exponent/fraction; no current native converter

Reset before returning to ordinary work

When the bit investigation is complete, exit BASE and execute **BRESET** the next time you begin BASE work. The reset returns word size 36, signed mode and non-wrapped behavior. It does not document your previous contract, so keep width, signedness and endianness with the calculation notes. A correct hex answer without those labels is an unfinished answer.

Appendix A. Techniques at a glance

Situation	Model / check	DM42 route
Reused parameter	Store the measured value with its unit recorded externally	STO 00, RCL 00, or a named variable
Complex phasor	$a + ib$ or $r \angle \theta$; confirm angle mode	MODES → RECT/POLAR; COMPLEX
Simultaneous equations	$Ax = b$; inspect residuals	MATRIX → SIMQ → MATA/MATB/MATX
Paired experimental data	Enter y first, x second; choose linear model	CLEAR → CLΣ; STAT → CFIT → MODL → LINP
Nonlinear equation	Express as $f(x) = 0$; test the root in original model	SOLVER with small labeled MVAR function
Definite integral	Supply limits, accuracy factor and a labeled integrand	f(x) → integrand → variable → LLIM/ULIM/ACC → ∫
Probability count	$\binom{n}{k} p^k (1-p)^{n-k}$; check total probability	PROB → COMB
Display scale	Choose useful engineering exponent; units stay your responsibility	DISP → ENG

Numerical checklist. (1) Write units and sign conventions; (2) estimate scale; (3) check angle mode; (4) compute at full internal precision; (5) substitute or derive an independent result; (6) round only when reporting. Solver residuals do not prove a unique root. A matrix solution may be numerically fragile if the equations are nearly dependent. A tiny integration error estimate concerns numerical approximation, not uncertain inputs. Formatting to six digits does not improve a two-digit measurement.

Selected exact menu spellings. The official SwissMicros quick guide gives CLΣ, ALLΣ, LINΣ, COMPLEX, SLOPE, YINT, CORR, SDEV, COMB, →POL, →REC, LLIM, ULIM, ACC, SIMQ, MATA, MATB, MATX, CPXRES, and REALRES. The calculator's $\angle$ in a polar complex display is intentionally distinguished here from mathematical $\angle$. This glossary records function names, not promises that every name is a dedicated physical key.

Appendix B. Constants and documentation

Constant or convention	Value used	Role
Standard gravity in terrestrial examples	9.81 m s^{-2}	Approximation, varies with place
Universal gas constant	$8.314462618 \text{ J mol}^{-1} \text{ K}^{-1}$	Ideal-gas example
Earth's standard gravitational parameter	$3.986004418 \times 10^{14} \text{ m}^3 \text{ s}^{-2}$	Approximate orbital example
Mean Earth radius	6371 km	Spherical approximation
Planck constant h	$6.62607015 \times 10^{-34} \text{ J s}$	Exact SI defining constant
Speed of light c	$299792458 \text{ m s}^{-1}$	Exact SI defining constant
Elementary charge e	$1.602176634 \times 10^{-19} \text{ C}$	Exact SI defining constant

Calculator sources (consulted September 2026). SwissMicros, *DM42 User Manual* v3.42, 22 September 2026: https://technical.swissmicros.com/dm42/doc/dm42_user_manual.html. SwissMicros, *DM42 Quick Reference Guide* v3.5.5, 22 April 2025: <https://technical.swissmicros.com/dm42/fat/HELP/dm42help.htm>. SwissMicros, *DM42 Functions* v2.0.2, 22 April 2025: <https://technical.swissmicros.com/dm42/fat/HELP/dm42functs.htm>. SwissMicros, *DM42 Firmware / Programs* (DM42 3.26, DMCP 3.29 listed): <https://technical.swissmicros.com/dm42/firmware/>. Thomas Okken, *Free42 documentation*, extensions updated 23 April 2025: <https://thomasokken.com/free42/>. Hewlett-Packard, *HP-42S Owner's Manual* (1988), and *HP-42S Programming Examples and Techniques* (1988), linked from <https://www.swissmicros.com/en/technical>. The HP literature is historical; DM42 menu information in this book follows the SwissMicros guide. The programmer addendum follows the DM42's documented Free42 3.2.8 feature set; Free42 3.3 BASE additions are discussed only to mark the compatibility boundary and are not presented as current DM42 commands.

Alphabetical index

Topic	Pages and sections	Topic	Pages and sections
Accuracy factor (ACC)	14 (§9.1)	LASTX	4 (§1.2)
AC impedance	10 (§6.1), 11 (§6.2)	Least squares	15 (§10.1)
ADC threshold	9 (§5.2)	Lens formula	8 (§4.3)
Amortisation	17 (§12.1)	LINE	15 (§10.1)
Angle mode	5 (§2.1), 16 (§11.2)	Magnetic reactance	10 (§6.1)
Arithmetic right shift	26 (§P.5)	Masks	24 (§P.4)
ASCII-compatible codes	33 (§P.11)	Matrices	11 (§7.1), 12 (§7.2)
Base conversion	20 (§13.6)	Measurement uncertainty	16 (§11.2), 16 (§11.3)
BASE integer arithmetic warning	35 (§P.13)	Midpoint rule	14 (§9.3)
Bayes' theorem	17 (§11.4)	Nodal analysis	11 (§7.1)
Binary32 and binary64	32 (§P.10)	Orbital period	7 (§3.3)
Binomial probability	16 (§11.1)	Overflow, signed and unsigned	23 (§P.3), 31 (§P.9)
Bit fields	24 (§P.4), 27 (§P.6)	Packing and unpacking	27 (§P.6)
Bit numbering and BIT?	24 (§P.4)	Parity	29 (§P.8)
Byte order	27 (§P.6)	Pendulum	8 (§4.6)
Capacitor charging	9 (§5.2), 12 (§8.1)	Phasors	10 (§6.1), 11 (§6.3)
Capacitor pulse sizing	19 (§13.2)	Photon energy	8 (§4.4)
Carry and borrow	26 (§P.5), 31 (§P.9)	Polar display	10 (§6.1)
Centre of mass	6 (§2.3)	Population count	29 (§P.8)
Character codes	33 (§P.11)	Projectile motion	5 (§2.1)
COMB	16 (§11.1)	Regression	15 (§10.1)
COMPLEX	10 (§6.1), 11 (§6.2)	Resonance	7 (§4.1), 10 (§6.1), 19 (§13.4)
Compound interest	17 (§12.2)	Rotates and ROTXY	26 (§P.5)
Conditioning and cancellation	13 (§8.3)	Root finding	12 (§8.1), 13 (§8.2)
Coordinate rotation	6 (§2.2), 12 (§7.2)	Sample standard deviation	15 (§10.2)
Correlation	15 (§10.1)	Shifts	26 (§P.5)
CRC-8 worked check	30 (§P.8)	Significant figures	5 (§1.3), 16 (§11.2)
Current, RMS	10 (§6.1), 11 (§6.3)	Signedness traps	28 (§P.7)
Date arithmetic	19 (§13.5)	Sign extension	28 (§P.7)
Decibels	4 (§1.1), 11 (§6.2)	SIMQ	11 (§7.1)
Degrees and radians	5 (§2.1), 16 (§11.2)	Snell's law	19 (§13.3)
Derivative, numerical	18 (§13.1)	Solver	12 (§8.1), 13 (§8.2)
Divider loading	4 (§1.2), 9 (§5.3)	Statistical sums	15 (§10.1), 15 (§10.2)
Effective annual rate	17 (§12.2)	Thévenin equivalent	9 (§5.3)
Electric power	9 (§5.1), 11 (§6.3)	Time constant	5 (§1.3), 9 (§5.2), 10 (§5.4)
Endianness	27 (§P.6)	Two's-complement arithmetic	22 (§P.1), 23 (§P.3)
Exponential decay	8 (§4.5), 14 (§9.1)	Uncertainty propagation	16 (§11.2), 16 (§11.3)
Finite difference	18 (§13.1)	Unsigned arithmetic	23 (§P.3)
Flywheel energy	7 (§3.2)	Voltage offset	15 (§10.1)
Gas law	8 (§4.2)	Weighted mean	16 (§10.3)
Hexadecimal	23 (§P.2)	Word size (WSIZE)	20 (§13.6), 22 (§P.1)
IEEE-754 field conversion	32 (§P.10)	Wrapped arithmetic (BWRAP)	23 (§P.3)
Integration	14 (§9.1), 14 (§9.2), 14 (§9.3)	XOR checksums	29 (§P.8)
Integer ranges	22 (§P.1)		